Università di Padova

Department of Mathematics "Tullio Levi-Civita"

Master's Degree in Computer Science

Empirical Profiling of Cloud-Based Healthcare Interoperability Gateways

Master Thesis

Candidate

Angeloni Alberto

Matricola 2131060

Supervisor

Prof. Vardanega Tullio

Academic Year 2025-2026

I declare that I have used generative artificial intelligence tools to support stylistic and structural revision and to prepare preliminary drafts for visual materials. I also declare that I have used generative tools to draft the initial content of the thesis; I then critically verified, corrected, and manually integrated that material against the cited sources. I reviewed, verified, and revised every artificial-intelligence-assisted output. I take full responsibility for the thesis's arguments, technical claims, source selection, citations, original contribution, editorial decisions, and final form.

Abstract

The digitalisation of community-based healthcare requires heterogeneous applications to exchange information through regional services whose behaviour must be assessed across complete transaction paths. In the context of SI.Ter (Sistema Informativo Territoriale), the Territorial Information System for the Healthcare Organisations of the Veneto Region, this thesis defines an experimental methodology for end-to-end performance profiling and differential overhead characterisation of cloud-based healthcare interoperability gateways. Documented regional workflows become reproducible workloads observed at the client boundary. The Request Dataset Generator (RDG) is a Python framework that creates synthetic Fast Healthcare Interoperability Resources (FHIR) R4 (Release 4) patients, composes ordered multi-step workflows, schedules configurable traffic, and concurrently executes independent flow occurrences. It supports the direct RVE (Regione Veneto) patient-registry transactions RVE-54 and RVE-55 and a gateway-mediated workflow in mock and live modes. Structured logs provide a traceable dataset for request- and flow-level latency, throughput, error-rate, concurrency, and scheduling metrics. Since internal gateway logs remain outside the authorised perimeter, the *Surrogate Observability Strategy* uses the `direct_vs_middleware` experiment to compare client-observed latency distributions for direct and mediated paths. Differences in means and selected percentiles express path-level displacement, not internal gateway time; routing, security, transformation, network, and retry effects may also contribute. The empirical campaign comprises 74 completed runs. As a proof of concept, it distinguishes transport from application outcomes, request from flow populations, direct from mediated paths, and successful from failed processing. The contribution is a profiling framework and a reproducible surrogate-observability protocol for constrained healthcare integration environments.

Contents

List of Figures

List of Tables

Acronyms and Abbreviations

APMS Application Profile Management System. 3

ATNA Audit Trail and Node Authentication. 7, 11

COT Centrale Operativa Territoriale. 1

FHIR Fast Healthcare Interoperability Resources. 5

FSE Fascicolo Sanitario Elettronico. x, 2, 7

HL7 Health Level Seven. 16

IAP Identity and Assertion Provider. 9

IHE Integrating the Healthcare Enterprise. 5

JSON JavaScript Object Notation. 5

JSONL JSON Lines. 16

JWT JSON Web Token. 7, 16

PEM Privacy-Enhanced Mail. 55

PKCS Public-Key Cryptography Standards. 55

RDG Request Dataset Generator. 10

RVE Regione Veneto. 5, 15

SAML Security Assertion Markup Language. 7, 16

SI.Ter Sistema Informativo Territoriale. 3

SSL Secure Sockets Layer. 55

XDS Cross-Enterprise Document Sharing. 4

Glossary

APMS A centralised system used by Azienda Zero to support user authentication and profile management for participating cloud-based applications. 3

ATNA An IHE integration profile that defines security measures for healthcare information systems, including node authentication, protected communications, and the generation and collection of audit records. 7, 11

COT A territorial coordination centre introduced in the Italian healthcare model to coordinate transitions among hospital, community, home-care, and other territorial services. 1

FHIR An HL7 standard for exchanging healthcare information electronically through modular resources, defined data models, and interoperable interfaces. 5

FSE The Italian *Fascicolo Sanitario Elettronico (FSE)*$_G$ infrastructure that makes health and social-care documents and data concerning an individual available to authorised actors. 2, 7

HL7 A family of standards for the exchange, integration, sharing, and retrieval of electronic health information. 16

IAP A regional security actor that authenticates a requesting application context and issues the identity assertion required by subsequent healthcare interoperability transactions. 9

IHE An international initiative that publishes integration profiles describing how established healthcare standards can be used together to support specific interoperability use cases. 5

JSON A text-based data-interchange format that represents structured data through objects, arrays, and primitive values. 5

JSONL A line-oriented data format in which each line is an independent JSON value, allowing structured event and execution records to be processed incrementally. 16

JWT A compact token format used to transmit claims between parties as a JSON object that can be digitally signed and, when required, encrypted. 7, 16

PEM A textual encoding format that represents cryptographic material, such as certificates and private keys, in Base64 blocks delimited by human-readable headers and footers. 55

PKCS A family of specifications for public-key cryptography; PKCS #12 defines a binary archive format that can contain a private key, its certificate, and an associated certificate chain. 55

RDG The software framework developed in this thesis to generate synthetic healthcare data, compose interoperability workflows, schedule workloads, execute requests, and collect structured experimental observations. 10

RVE The acronym used as a prefix for interoperability transactions defined for the Veneto regional healthcare ecosystem. 5, 15

SAML An XML-based standard for exchanging authentication and authorisation assertions between identity providers and service providers. 7, 16

SI.Ter The Veneto regional initiative supporting territorial-care applications and their integration with local and regional healthcare systems. 3

SSL A legacy family of cryptographic protocols for protected network communication and the predecessor of Transport Layer Security; contempo-

rary software may retain SSL in class or module names while negotiating Transport Layer Security. 55

XDS and XDS.b An IHE integration profile for registering, discovering, and retrieving clinical documents across healthcare organisations. XDS.b denotes the revised profile variant used by the interoperability architecture considered in this thesis. 4

Chapter 1

Problem Statement

This chapter defines the research problem by connecting the regulatory mandate for territorial-care coordination to the heterogeneous SI.Ter integration architecture. It first establishes the institutional and technical context, then formalises the resulting continuity and observability constraints, and finally derives the methodological response, research objectives, and contributions addressed by the thesis.

1.1 Regulatory and Technical Context

1.1.1 Territorial Coordination

DM (Decreto Ministeriale / Ministerial Decree) 77/2022 defines territorial care as a coordinated network of services rather than as a collection of isolated clinical encounters. Its organisational model includes the District, the Casa della Comunità, the COT (Centrale Operativa Territoriale / Territorial Coordination Centre), the 116117 operational centre, home-care services, community hospitals, and telemedicine-supported services[1]. The decree therefore assigns digital information exchange an operational role: coordination across care settings depends on information that remains available when responsibility for a patient moves between hospital departments, territorial services, general practitioners, and home care. The COT makes this dependency explicit because it coordinates transitions among care settings and connects actors that belong to different organisational and technical domains. Its work requires current patient-identification data, care-transition information, clinical documentation,

[1]Ministero della Salute. *Regolamento recante la definizione di modelli e standard per lo sviluppo dell'assistenza territoriale nel Servizio sanitario nazionale.* Ministerial decree Decreto 23 maggio 2022, n. 77. Italian Official Gazette, General Series no. 144, 22 June 2022. Ministero della Salute, May 23, 2022, Allegato 1, pp. 12, 20–21.

and communication with local and regional services[2]. A protected discharge, for example, does not end when a hospital records the discharge decision: it continues through the transmission, assessment, assignment, and activation activities that make territorial care effective. The organisational process is consequently continuous even when the supporting applications are not.

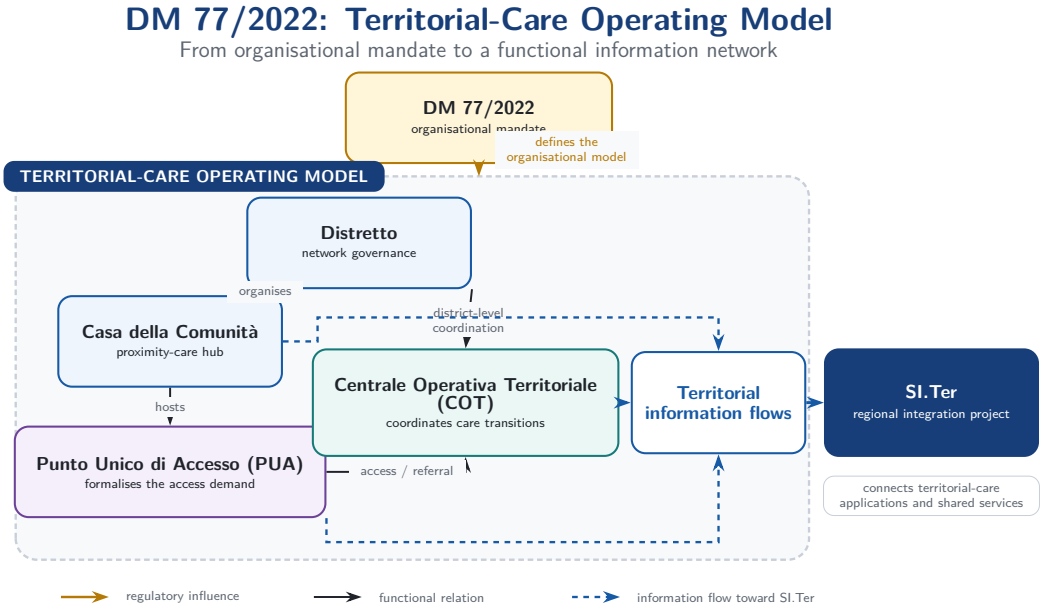


Figure 1.1: Concept of the territorial-care operating model defined by DM 77/2022. Source: Adapted from the Ministerial Decree 77/2022.

This continuity requirement operates within a regulated data environment. Health data are a special category of personal data under Article 9 of European Union Regulation 2016/679, and their processing remains subject to data protection by design and by default and to measures appropriate to risk under Articles 25 and 32[3]. The FSE (Fascicolo Sanitario Elettronico), or Electronic Health Record, translates the same principle into role-based authorisation, pe-

[2]Ministero della Salute, *Regolamento recante la definizione di modelli e standard per lo sviluppo dell'assistenza territoriale nel Servizio sanitario nazionale*, Allegato 1, pp. 20–21.

[3]European Parliament and Council of the European Union. *Regulation (EU) 2016/679 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data*. Regulation (EU) 2016/679. Official Journal of the European Union, L 119. European Union, Apr. 27, 2016, pp. 1–88. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj (visited on 07/29/2026), arts. 9, 25 and 32.

riodic verification of access profiles, secure communication, operational traceability, and audit-log mechanisms[4]. DM 77/2022 thus creates a dual technical requirement: information must cross organisational boundaries to support the COT, yet each crossing must preserve identity, authorisation, confidentiality, and traceability. Satisfying both conditions requires the territorial-care model to interoperate with the heterogeneous systems already present in the regional ecosystem.

1.1.2 Heterogeneity in the Territorial Information System

The regional programme provides the Veneto project context in which the organisational mandate becomes an integration problem. Regional documentation defines SI.Ter as a programme that connects territorial-care application domains, shared services, and pre-existing local and regional systems; it does not assume their replacement by a single greenfield platform[5]. The procurement framework accordingly requires cooperation across distinct ownership boundaries while preserving the processes and information assets already managed by Healthcare Organisations[6]. The COT model therefore depends on an architecture whose components remain heterogeneous in responsibility, interface, and data representation. The integration perimeter includes APMS (Application Profile Management System) for authentication and user-profile management

[4]Ministero della Salute. *Fascicolo sanitario elettronico 2.0.* Ministerial decree Decreto 7 settembre 2023. Italian Official Gazette, General Series no. 249, 24 October 2023, as subsequently amended. Ministero della Salute, Sept. 7, 2023. URL: https://www.gazzettaufficiale.it/eli/id/2023/10/24/23A05829/sg (visited on 07/29/2026), art. 25.

[5]Regione del Veneto, Area Sanità e Sociale. *Modello di Governo per l'implementazione del Sistema Informativo Territoriale (SI.Ter) nelle Aziende ed Enti del Servizio Sanitario Regionale della Regione del Veneto.* Regional decree Decreto n. 11922. Regione del Veneto, Mar. 16, 2026, pp. 1–3.

[6]Azienda Zero. *Realizzazione di un Sistema Informativo Territoriale per gli Enti Sanitari della Regione Veneto.* Technical specifications. Specific procurement, Digital Healthcare – Clinical-Care Information Systems, ID 2365, Lot 3; date derived from file metadata. Azienda Zero, July 9, 2025; Raggruppamento Temporaneo di Imprese guidato da AlmavivA. *Sistema Informativo Territoriale per gli Enti Sanitari della Regione del Veneto – Lotto 3.* Technical report. Date derived from file metadata. Raggruppamento Temporaneo di Imprese guidato da AlmavivA, Sept. 19, 2025.

in cloud-based applications[7], Anagrafe Zero for patient identification and registration, local Admission, Discharge, and Transfer systems, and the FSEr (Fascicolo Sanitario Elettronico regionale) document infrastructure. Within the document path, XDS (Cross-Enterprise Document Sharing) repositories publish and retrieve clinical documents according to the regional Affinity Domain, while their indexing makes those documents available to the regional record. The documented SI.Ter architecture also connects context-call services, digital signature providers, prescription services, local clinical applications, and other shared components[8]. Within the integration stack represented by the thesis, Scryba Sign instantiates the signature-provider role and exposes its services through SOAP (Simple Object Access Protocol) interfaces described by WSDL (Web Services Description Language) contracts. The component handles FEA (Firma Elettronica Avanzata) and remote digital signatures; when the selected certificate and trust-service conditions satisfy the applicable eIDAS (electronic identification, authentication and trust services) requirements, the latter supports FEQ (Firma Elettronica Qualificata). It produces CAdES (CMS Advanced Electronic Signatures), PAdES (PDF Advanced Electronic Signatures), and XAdES (XML Advanced Electronic Signatures) envelopes and applies timestamping in the TimeStampedData (TSD) format. This classification identifies the supported integration surface and does not by itself certify the legal qualification of every configured signature[9]. The technological

[7]DXC Technology. *Specifiche di integrazione autenticazione APMS*. Technical specification. Version 1.4.1. Azienda Zero, Nov. 22, 2024, Version 1.4.1, Section 1.

[8]Raggruppamento Temporaneo di Imprese guidato da AlmavivA, *Sistema Informativo Territoriale per gli Enti Sanitari della Regione del Veneto – Lotto 3*, Sections 2.1.2 and 2.1.4.

[9]Medas S.r.l. *Scryba Sign 3.x Developer's Guide*. Developer's guide. Version 16. Restricted internal document. Medas S.r.l., Apr. 10, 2024, pp. 7, 10, 28, and 89; European Parliament and Council of the European Union. *Regulation (EU) No 910/2014 on Electronic Identification and Trust Services for Electronic Transactions in the Internal Market*. Consolidated text of 18 October 2024. July 23, 2014. URL: https://eur-lex.europa.eu/eli/reg/2014/910/2024-10-18/eng (visited on 08/05/2026), Articles 3 and 25; Adriano Santoni. *RFC 5544: Syntax for Binding Documents with Time-Stamps*. RFC Editor. Feb. 2010. DOI: 10.17487/RFC5544. URL: https://www.rfc-editor.org/info/rfc5544/ (visited on 08/05/2026), Section 2.

gap becomes visible in the interaction models exposed by the two principal data domains. Anagrafe Zero provides resource-oriented REST (Representational State Transfer) interactions over HTTP (Hypertext Transfer Protocol), based on HL7 (Health Level Seven) FHIR (Fast Healthcare Interoperability Resources) R4 (Release 4), for RVE (Regione Veneto) operations such as patient search and identifier assignment. FHIR admits both XML (Extensible Markup Language) and JSON (JavaScript Object Notation) representations, but the applicable regional interactions specify FHIR XML; the contrast is therefore not a simple XML-versus-JSON conversion[10]. The FSEr document infrastructure instead applies IHE (Integrating the Healthcare Enterprise) XDS.b transactions through SOAP (Simple Object Access Protocol) Web Services[11]. Both families can use HTTP over TCP (Transmission Control Protocol) and protect the channel through mutual Transport Layer Security (mTLS). The mediation challenge therefore lies above this common transport substrate: SI.Ter must reconcile resource-oriented REST operations with message-oriented SOAP transactions and map patient identifiers, document metadata, security contexts, error semantics, and workflow dependencies without changing their clinical meaning. Interface connectivity alone is insufficient because syntactically valid forwarding does not guarantee semantic equivalence between the two application contracts. An end-to-end care transition can consequently traverse several systems before the receiving actor obtains the information required to continue the process. The solution architecture mediates these interactions through application interfaces and an interoperability layer. The layer mediates RESTful FHIR and SOAP-based IHE XDS.b interactions with ebXML (Electronic Business using

[10]Elena Vio et al. *Specifiche tecniche – Anagrafe Zero.* Technical specification. Version 2.6. Consorzio Arsenàl.IT, Jan. 9, 2024, Sections 1.5.2 and 2.5.2.

[11]IHE International. *IHE IT Infrastructure Technical Framework, Volume 1: Cross-Enterprise Document Sharing (XDS.b), Revision 20.2.* Nov. 11, 2025. URL: https://profiles.ihe.net/ITI/TF/Volume1/ch-10.html (visited on 07/22/2026); CNR (ICAR) e Dipartimento per la Trasformazione Digitale. *Specifiche tecniche per l'interoperabilità tra i sistemi regionali di FSE: Affinity Domain Italia.* Technical specification. Version 2.6.3. CNR (ICAR) e Dipartimento per la Trasformazione Digitale, Oct. 31, 2025.

eXtensible Markup Language) metadata, while managing asynchronous orchestration, transformation, errors, and interface lifecycles[12]. The integration stack connects heterogeneous services, ranging from context-call providers to digital signature engines, within a regulated regional perimeter. Figure 1.2 illustrates the architectural layers of this environment, highlighting the mediation challenge: the interoperability layer must reconcile the resource-oriented REST interactions of Anagrafe Zero with the message-oriented SOAP transactions required by the FSEr document infrastructure. This mediation, protected by mutual Transport Layer Security (mTLS), ensures that clinical meaning and security contexts are preserved as information crosses organisational boundaries. The specific standards and functional roles governing these layers are further detailed in Table 1.1.

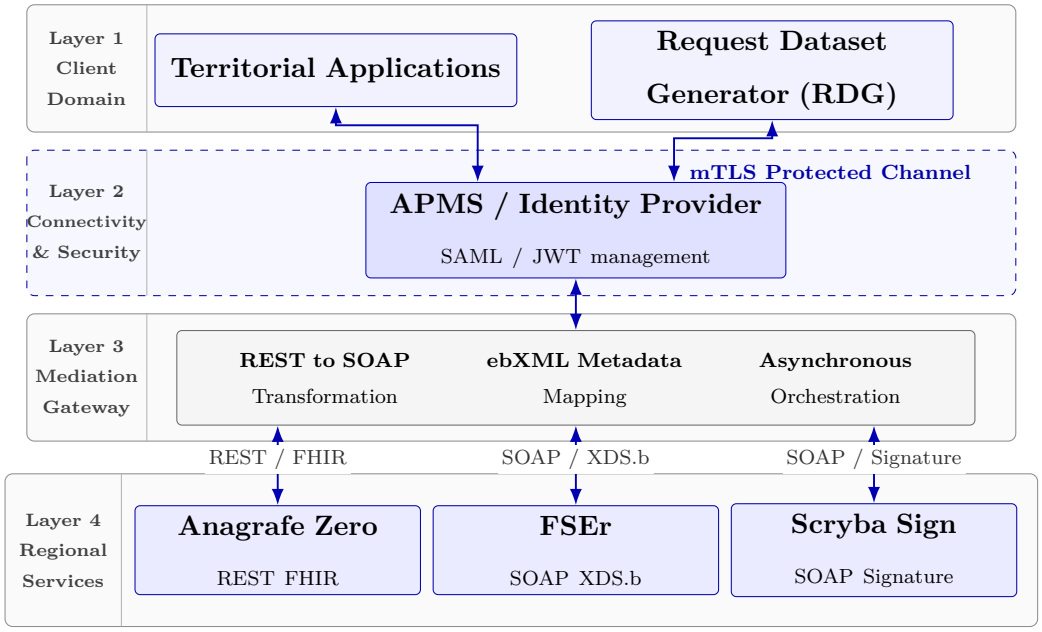


Figure 1.2: Architectural Layers of the SI.Ter Integration Stack. Source: Author's elaboration based on regional technical specifications.

[12]Raggruppamento Temporaneo di Imprese guidato da AlmavivA, *Sistema Informativo Territoriale per gli Enti Sanitari della Regione del Veneto – Lotto 3*, Sections 2.1.2 and 2.1.4.

Table 1.1: Summary of key standards, including regulatory constraints, and their functional roles in the SI.Ter integration stack. Source: Author's synthesis based on the cited regulatory, regional, and interoperability specifications.

Standard	Protocol or format	Role in the project
HL7 FHIR R4 Health Level Seven Fast Healthcare Interoperability Resources, Release 4	HTTP/REST; `Patient` and `Bundle` resources represented in FHIR XML, with JSON where required by the applicable interaction	Supports patient-registry interactions with Anagrafe Zero, including patient search, identifier assignment, and demographic updates such as [RVE-54] and [RVE-55].
IHE XDS.b Integrating the Healthcare Enterprise Cross-Enterprise Document Sharing	SOAP/XML messages, ebXML metadata, and the ITI (IT Infrastructure) transactions ITI-41, ITI-18, and ITI-43	Supports publication, metadata indexing and query, and retrieval of clinical documents through the FSEr Document Repository and Document Registry.
SAML (Security Assertion Markup Language) 2.0 / JWT (JSON Web Token)	XML assertions and compact RFC (Request for Comments) 7519 tokens carried by the transaction-specific SOAP or HTTP/REST exchanges	Establishes and propagates the security context: [RVE-121] uses the SAML assertion to obtain a JWT, which is then used by [RVE-130] for the contextual service call.
GDPR (General Data Protection Regulation) / FSE (Fascicolo Sanitario Elettronico) 2.0	Data Protection by Design and by Default constraints	Establishes design constraints for differentiated access profiles and operation traceability. In the regional stack, IHE ATNA (Audit Trail and Node Authentication) implements audit logging. Authorisation checks and audit-record generation constitute non-bypassable processing in a compliant path; their cost therefore remains part of the measured end-to-end performance.
eIDAS; CAdES, PAdES, and XAdES Electronic-signature legal framework and signature-envelope profiles	SOAP Web Services; CMS, PDF, and XML signature envelopes; TimeStampedData temporal evidence	eIDAS supplies the legal classification of electronic signatures, while the envelope and timestamp formats define the Scryba Sign integration artefacts processed by the calling application.

These capabilities solve the connectivity problem only at the interface level. They do not by themselves establish whether a multi-system care process re-

mains reconstructable from its initial request to its final outcome. Each mediation function introduces a boundary at which identifiers, security contexts, payloads, or timing information may change. Authentication can be a prerequisite for a patient query; the patient identifier returned by that query can become an input to document search; a retrieved document can affect a later care decision. The integration gap is therefore not the absence of interfaces, but the absence of a single observation point that represents the meaning, ordering, and outcome of all interactions involved in a care workflow. This gap turns the heterogeneity of SI.Ter into a problem of information continuity and end-to-end traceability.

1.2 Continuity and Observability

1.2.1 Workflow Traceability

Territorial care crosses organisational, application, and information boundaries at the same time. Organisational fragmentation arises because a single care pathway assigns responsibilities to hospital departments, the COT, territorial services, general practitioners, and home-care providers. Application fragmentation arises because these actors use software governed by different organisations. Information fragmentation arises because identity, clinical, administrative, and workflow data reside in registries, local applications, repositories, and regional platforms. Interoperability connects these domains, but the care process remains traceable only when the connections preserve the relationships among actors, data, and ordered activities. The protected-discharge workflow illustrates the resulting dependency chain. A hospital operator identifies the patient, assembles the information required for the referral, selects a proposed territorial setting, and submits the request to the competent COT. The COT receives and evaluates the request, coordinates the actors involved, and advances the case through explicit workflow states. The protected-admission workflow applies an analogous coordination pattern in the opposite direction,

from a territorial setting towards a hospital structure[13]. These processes are multi-actor workflows rather than sequences of unrelated service calls: their correctness depends on the availability and interpretation of outputs produced by earlier activities.

The upper branch in Figure 1.3 makes the identity dependency concrete. Within `FLOW_PATIENT_QUERY`, RDG executes [RVE-1.b] before the [RVE-54] Patient Query. Through the first transaction, the regional IAP (Identity and Assertion Provider) generates and returns a SAML 2.0 identity assertion; the execution engine extracts its required Base64 representation, stores it in the context of the same flow occurrence, and resolves the corresponding placeholder in the `Authorization` header of [RVE-54] before sending the query. The resulting header carries the assertion with the `IHE-SAML` authentication scheme, as required by the regional security profile[14][15]. RDG therefore represents regional security-session state as explicit per-occurrence dependency data rather than reducing it to the state of the underlying HTTP connection. If [RVE-1.b] fails, or if the required assertion cannot be extracted or its placeholder remains unresolved, the engine does not execute a semantically valid patient query and records the workflow as incomplete. This behaviour makes assertion acquisition and propagation part of the tested workflow. An isolated request can exercise [RVE-54] only after receiving a valid security context from outside the test and therefore cannot evaluate the identity hand-off on which the regional interaction depends.

A technical evaluation based on isolated endpoints cannot represent this dependency. A successful authentication request does not prove that the resulting security context is accepted by the next service; a successful patient query does not prove that the returned identifier reaches document search; and a set of

[13]Raggruppamento Temporaneo di Imprese guidato da AlmavivA, *Sistema Informativo Territoriale per gli Enti Sanitari della Regione del Veneto – Lotto 3*, Sections 2.1.5 and 2.4.1.

[14]Mauro Zanardini et al. *Infrastruttura di sicurezza GDL-O Sicurezza.* Technical specification. Version 2.14. Consorzio Arsenàl.IT, June 28, 2024, Section 4.2.

[15]Vio et al., *Specifiche tecniche – Anagrafe Zero*, Section 1.7.

individually successful calls does not prove that the workflow reaches its intended terminal state. The relevant unit of analysis is therefore one occurrence of an end-to-end workflow, considered both through its executed interactions and through its overall outcome and completion time. This unit preserves the functional meaning that disappears when measurements are aggregated only by endpoint. End-to-end reconstruction also requires stable correlation identifiers, ordered timestamps, transport and application outcomes, and explicit data dependencies between steps. Without this evidence, a failure can be located at the client boundary but cannot be associated reliably with the workflow state that precedes or follows it. The continuity problem consequently becomes an observability problem: the evaluation method must reconstruct the external behaviour of the distributed workflow, while the regional security perimeter limits which internal evidence the experiment can inspect.

Figure 1.3 maps this requirement to the Request Dataset Generator (RDG) flow model, in which each correlated occurrence retains both business outputs and the security context required by downstream transactions.

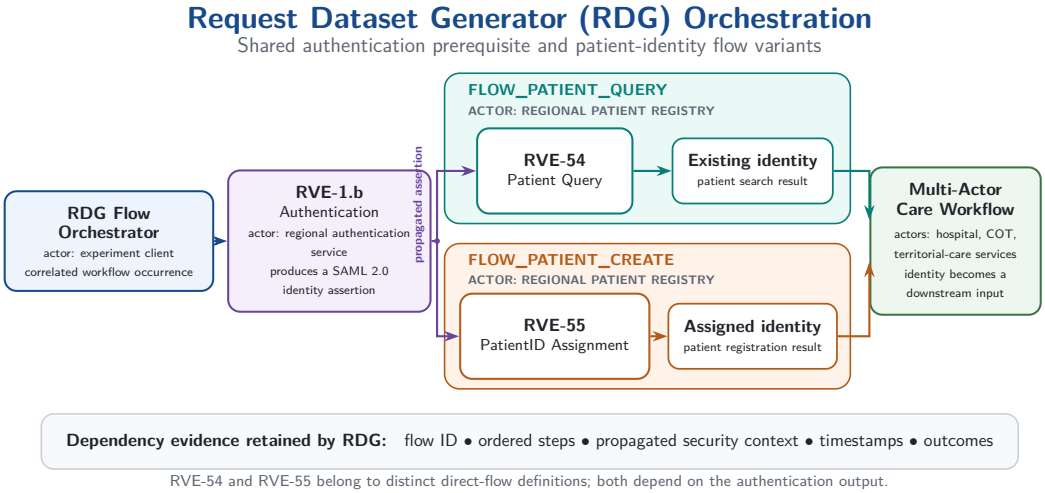


Figure 1.3: Request Dataset Generator orchestration of the [RVE-1.b] authentication output into regional patient-identity flows. Source: Author's elaboration based on the framework implementation and regional specifications.

1.2.2 The Authorised Observation Boundary

Regional security specifications make authentication, authorisation, secure transport, and auditability integral to the integration architecture. Operators and applications act within an authenticated and authorised context, and assertions or tokens can become prerequisites for later services[16]. The architecture also provides audit mechanisms for production accountability; the regional security specification states that participating actors generate audit records and refers their detailed structure to the applicable ATNA (Audit Trail and Node Authentication) profile[17]. The existence of this production telemetry does not make it part of the evidence available to every consumer or experiment, because access remains governed by operational roles and security policy. Within the authorised perimeter of this thesis, internal APMS telemetry and telemetry from the mediated gateway path are not made available to the experiment. This statement describes the research boundary; it does not claim that the platforms fail to produce logs or that regional policy universally prohibits their inspection. Real patient data, credentials, private endpoints, confidential deployment parameters, and production audit records likewise remain outside the disclosed dataset. Production services are neither instrumented by the thesis nor subjected to intrusive load.

These restrictions, summarised in Table 1.2, preserve the applicable security boundary but remove the internal timestamps required for a causal decomposition of gateway processing. The observable interval consequently extends from the client sending a request to the client receiving the corresponding response. It can include client-side processing, authentication, network transit, gateway mediation, downstream service execution, and response propagation. A latency measured across this interval characterises the path as a whole; it cannot

[16]DXC Technology, *Specifiche di integrazione autenticazione APMS*, Version 1.4.1; Zanardini et al., *Infrastruttura di sicurezza GDL-O Sicurezza*; Gregorio Canal and Gianluca Pavan. *Specifiche tecniche chiamata di contesto RVE*. Technical specification. Version 1.3. Consorzio Arsenàl.IT, July 18, 2025.

[17]Zanardini et al., *Infrastruttura di sicurezza GDL-O Sicurezza*, Section 4.3.15.

attribute a delay to a specific gateway component without additional internal evidence. Treating the entire difference as gateway processing therefore exceeds the available data and violates the distinction between observation and causal explanation. This limitation determines the methodological requirement. The experiment needs a controlled external layer that represents complete workflows, fixes the workload and observation rules, records client-visible events, and compares direct with gateway-mediated execution at the same declared boundary. Such a layer does not replace production telemetry. It provides surrogate evidence for the effect visible outside the protected platform and makes every inference conditional on the comparability of the observed paths. The resulting research question is: *how can documented healthcare interoperability workflows be translated into reproducible, observable, and measurable end-to-end workloads, and how can the apparent overhead associated with gateway mediation be estimated differentially when internal platform telemetry is outside the authorised observation perimeter?*

Table 1.2: Comparison between internal production evidence and external experimental observation. Source: Author's elaboration based on the cited regional security specifications and the RDG evidence model.

Dimension	**Authorised Observation Boundary (Internal)**	**Experimental Observation Perimeter (RDG)**
Data access	Real clinical data and ATNA audit records remain production-side and are accessible only to authorised operational roles.	RDG uses synthetic FHIR patients; real health data and production audit records remain outside the dataset.
Granularity	Component-level timestamps and traces remain protected; their availability depends on production instrumentation and authorised operator access.	RDG measures client-observed mediated-path latency $L_{\mathrm{med}}(q)$ and the differential $\Delta L(q) = L_{\mathrm{med}}(q) - L_{\mathrm{dir}}(q)$ for the declared statistic q.
Outcome insight	Internal traces can associate a failure with a component only when the corresponding production evidence is available.	RDG distinguishes the HTTP transport outcome from application completion and semantic rejection visible in the response or output extraction.
Telemetry	APMS, mediated-gateway, and production ATNA logs remain inaccessible to the experiment.	RDG produces correlated JSONL events for requests, responses, and completed workflow occurrences.
Overhead attribution	Direct gateway measurement requires aligned ingress, internal-component, and egress timestamps, which lie outside the experimental perimeter.	ΔL measures an aggregate path-level effect that can include routing, security, transformation, downstream-service, and retry costs; it does not decompose APMS or gateway processing.

1.3 Methodological Response

1.3.1 Surrogate Observability

RDG answers this question as the technical instrument of a *Surrogate Observability Strategy*. The strategy translates documented healthcare interactions into synthetic and executable workloads, observes them at the client boundary, and compares controlled direct and gateway-mediated paths. Its central differential mechanism is the `direct_vs_middleware` experiment. The current implementation groups direct patient-query and patient-creation flows, contrasts their client-observed end-to-end completion-time distributions with the mediated patient-query flow, and reports differences for the mean and the 50th, 95th, and 99th percentiles. These differences estimate *path-level overhead*; they do not decompose the internal execution time of the gateway.

Observation in this strategy is not synonymous with measuring network latency. The client-observed interval covers the complete request–response path, while RDG records the HTTP-level outcome and the application outcome as distinct evidence. An HTTP status code 200 confirms that the exchange returns a successful HTTP response, but it does not by itself prove that the payload contains the required output or that the workflow can advance. The execution engine therefore marks a step as failed when a mandatory output cannot be extracted, even if the response status is 200, and propagates that failure to the flow-level outcome. The same observation model retains service-reported semantic rejections, such as an [RVE-55] status code 422 associated with FHIR validation, without inferring the specific violated constraint when payload-level evidence is unavailable. The sanitised evidence set illustrates the first distinction through RVE-TOKEN responses that carry status code 200 but remain application failures because the required token is not extracted. [RVE-121] provides a complementary limit: its observed status code 200 demonstrates the current HTTP exchange, whereas evidence item `E-RVE-121-001` does not establish that a specific correction causes a transition from an earlier status code

400 to 200. Surrogate observability therefore separates HTTP-level evidence, application completion, and causal claims instead of treating every successful HTTP response as an equivalent logical outcome.

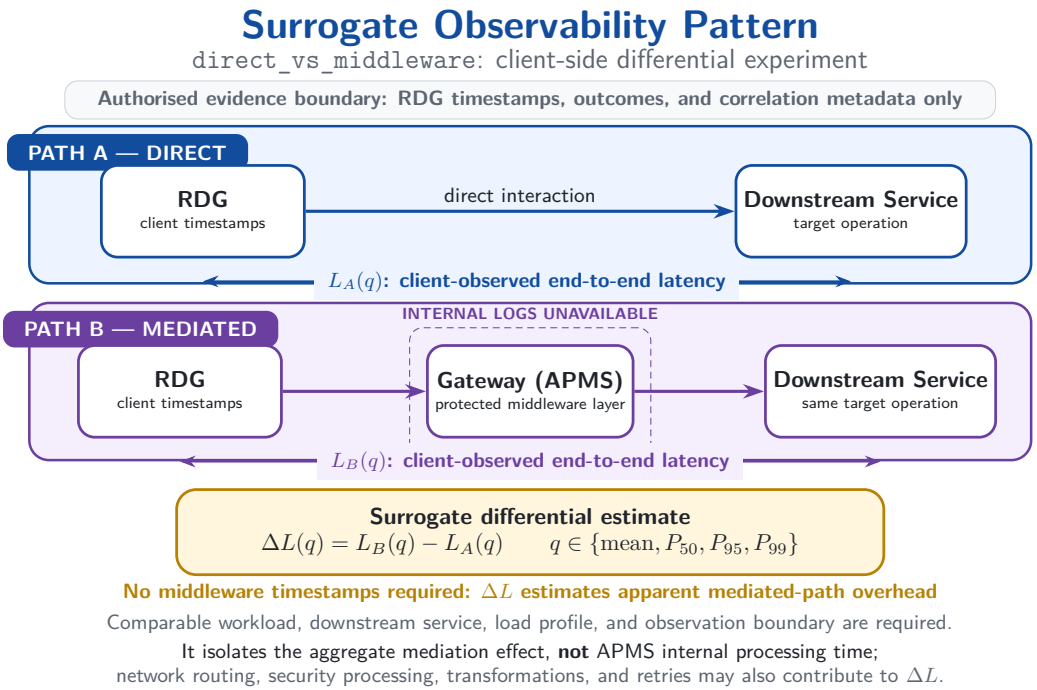


Figure 1.4: Surrogate observability pattern for the `direct_vs_middleware` experiment. Source: Author's elaboration based on the RDG experimental design.

The framework preserves the process semantics required by that comparison. It generates synthetic FHIR R4 patients, composes selected RVE (Regione Veneto) transactions into ordered multi-step flows, resolves outputs that become inputs to later steps, and assigns traffic occurrences to configurable timelines. Within each workflow occurrence, dependent steps execute sequentially; independent occurrences can execute concurrently through Python `asyncio`. Structured client-side events retain flow identity, step identity, timing, transport status, application outcome, and scheduling context. The resulting observation layer reconnects the functional workflow described by the SI.Ter documentation with the measurements available outside the protected gateway. The strategy covers selected workflow families related to regional authenti-

cation, patient identity, application context, document exchange, and digital signature. Its standards perimeter includes HL7 (Health Level Seven) FHIR R4 resources, RVE specifications, IHE XDS.b transactions, SAML (Security Assertion Markup Language) assertions, and JWT (JSON Web Token) tokens. Synthetic patients replace real health information, while mock and authorised live modes separate controlled verification of the pipeline from observation of remote paths. This scope supports a reproducible performance method without claiming clinical effectiveness, legal certification, exhaustive coverage of SI.Ter, or successful validation of every implemented transaction.

1.3.2 Scope, Objectives, and Contributions

The method pursues four verifiable objectives:

1. interpret the standards, protocols, security mechanisms, and integration architectures that define the selected healthcare workflows;

2. formalise protected discharge, protected admission, and the associated interoperability interactions in terms of actors, preconditions, ordered steps, and information dependencies;

3. provide a configurable framework that translates those dependencies into synthetic, executable, and observable workloads;

4. define measurements that support differential comparison between direct and gateway-mediated paths.

The objectives establish an evidence-producing method and artefact; they neither promise a performance improvement nor predetermine the experimental outcome. RDG makes the objectives inspectable through a run-specific artefact chain. The chain comprises a resolved configuration, a synthetic dataset, a traffic timeline, a JSONL (JSON Lines) execution log, a normalised analytical dataset, metric reports, tabular summaries, and optional graphs. The analytical layer separates request latency, workflow completion time, throughput, error

rate, concurrency, scheduling delay, and log completeness. Artefact presence establishes software output coverage, whereas only an executed and quality-checked scenario provides experimental evidence. This distinction keeps the methodological contribution independent from any unsupported claim about observed performance. The contribution is consequently a reproducible procedure for profiling healthcare interoperability paths when the internal gateway remains outside the observation perimeter. The analytical component reconstructs the meaning and dependencies of the selected regional flows; the software component generates, executes, correlates, and measures their occurrences; the differential component quantifies the externally visible difference between declared direct and mediated paths. Together, these components close the causal chain that begins with the coordination mandate of DM 77/2022: the COT requires cross-setting information continuity, SI.Ter realises that requirement through heterogeneous integrations, distributed workflows create an end-to-end traceability problem, security constrains internal observation, and RDG supplies the surrogate evidence needed for controlled evaluation. The organisation of the thesis preserves the same progression. Chapter 2 examines the architectural, interoperability, workload, and performance foundations required to interpret the solution space. It also states the requirements, use cases, transactions, and ex ante evaluation criteria supported by the available sources. Chapter 3 presents RDG from the inside: framework design and implementation, data and workload generation, execution and observability, the experimental protocol, and the results produced by declared runs. Chapter 4 closes the traceability chain from objectives and artefacts to criteria, key performance indicators, and results; it assesses the contribution, states the limits of the evidence, and derives future directions from the actual findings.

Chapter 2

Solution Space

This chapter evaluates the methodological alternatives available for performance profiling under a constrained observation perimeter. It establishes the statistical and distributed-systems foundations of the selected strategy, maps healthcare and cryptographic dependencies to experimental controls, defines the semantics and admissibility gates of the resulting evidence, and concludes by comparing generic and domain-specific profiling approaches.

2.1 Scientific Basis of the Profiling Strategy

2.1.1 Experimental Alternatives and Validity

The methodological problem is not the generation of a large number of requests. It is the production of interpretable performance evidence for stateful healthcare workflows when the experimenter controls the initiating client but does not control the complete regional execution path. The relevant unit of analysis is therefore an information-dependent workflow rather than an isolated endpoint. A valid experiment preserves the order in which security material, patient identifiers, context information and application outcomes become available. It also distinguishes the load offered to the system from the work that the system completes.

Four experimental strategies define the solution space. Operational observation offers high ecological fidelity, but it does not isolate the offered load from concurrent changes in users, services, routing, and infrastructure. An isolated protocol benchmark offers strong repeatability, but it removes the state transitions that determine whether a regional workflow reaches its terminal outcome. Trace replay can preserve an observed temporal pattern, but it requires an authorised trace whose state, ordering, and dependencies support faithful reconstruction. Controlled synthetic workflows make composition, timing,

concurrency, and randomisation explicit, but their resemblance to production traffic requires independent validation.

Zhu, Chen, and Chiueh provide the decisive methodological distinction. Their trace-replay study treats the input trace, the initial system state, and the dependencies among operations as necessary conditions for accurate replay. The authors also show that temporal or spatial scaling remains credible only when it does not violate those dependencies[1]. Their result does not establish the universal superiority of replay over synthetic benchmarks. It establishes that replay validity depends on evidence that exists before the experiment begins. In the regional case examined by this thesis, no authorised production trace, complete state reconstruction, sanitisation procedure, or replay agreement enters the available evidence perimeter. Consequently, trace replay is not an admissible experimental option. Within this perimeter, controlled synthetic workflows constitute the only strategy that simultaneously preserves internal validity, data minimisation, and repeatability. The conclusion is conditional rather than absolute: synthetic execution is the only controllable route among the evidence sources that the experimenter is authorised to use. It does not reproduce an unknown operational distribution by assumption. Instead, it creates matched experimental populations whose differences follow from declared factors. This distinction prevents the absence of production evidence from becoming an implicit claim of production representativeness.

Table 2.1 separates the scientific role of each alternative from the claim that it can support.

[1]Ningning Zhu, Jiawu Chen, and Tzi-cker Chiueh. "TBBT: Scalable and Accurate Trace Replay for File Server Evaluation". In: *4th USENIX Conference on File and Storage Technologies (FAST 05)*. San Francisco, CA: USENIX Association, Dec. 2005. URL: https://www.usenix.org/conference/fast-05/tbbt-scalable-and-accurate-trace-replay-file-server-evaluation (visited on 07/29/2026).

Table 2.1: Experimental alternatives and the claims that each strategy can support within the available evidence perimeter. Source: Author's synthesis based on Zhu et al. (2005).

Strategy	**Primary strength**	**Admissible claim**
Operational observation	Natural service mix and environmental realism.	Descriptive evidence for the observed environment when access and provenance are complete.
Isolated protocol benchmark	Repeatable control of one exchange.	Baseline for one contract, not completion of a stateful healthcare flow.
Trace replay	Preservation of captured ordering and timing when state and dependencies remain reconstructable.	Reproduction of an authorised trace within its capture and scaling assumptions.
Synthetic workflow	Explicit flow mix, timing, seed, concurrency, and data policy.	Internally valid comparison of declared scenarios; external representativeness remains unproven.

The resulting validity model separates four questions. Construct validity concerns whether the executable workload preserves the documented meaning and prerequisites of each transaction. Internal validity concerns whether matched scenarios differ only in the intended factor. External validity concerns whether an independent regional source supports the configured temporal profile, payload classes, and flow mix. Conclusion validity concerns whether repeated observations, explicit populations, and visible missing data support the reported comparison. Reproducibility therefore does not imply representativeness, and a realistic observation does not imply causality when uncontrolled environmental changes coexist with the measured path.

2.1.2 Synthetic Workflows as Experimental Instruments

A synthetic workflow has scientific value only when it acts as an experimental instrument rather than as a collection of fabricated requests. Its structure derives from authoritative transaction and process specifications; its data remain synthetic; its schedule derives from declared stochastic or deterministic rules; and its outcomes remain classified at transport, application, and complete-flow levels. These properties make the independent variables inspectable and the dependent variables reproducible without requiring real patient records or security artefacts.

The strategy distinguishes *what* executes from *when* execution starts. The flow mix specifies semantically distinct workflows. The timeline assigns their planned start times. A homogeneous Poisson process supplies one reproducible baseline. For intensity $\lambda > 0$, the arrival count in an interval of duration t follows

$$\Pr[N(t) = k] = \frac{(\lambda t)^k e^{-\lambda t}}{k!}, \qquad k = 0, 1, 2, \ldots. \tag{2.1}$$

A non-stationary profile replaces the constant rate with $\lambda(t)$, for which

$$\mathbb{E}[N(a, b)] = \int_a^b \lambda(u)\, du. \tag{2.2}$$

Piecewise rates express controlled ramps and peaks, while parameterised bursts express short concentrations around a baseline. These models define experimental hypotheses; they do not estimate Veneto clinical demand in the absence of an authorised operational series.

The synthetic strategy preserves a second separation between flow arrivals and request arrivals. One flow can expand into different numbers of requests, carry different payload classes, and terminate at different points. Equal flow rates therefore do not imply equal request rates or equal remote work. A shift toward document processing changes transferred bytes and downstream computation; a shift toward authentication and context access changes the number of dependent steps and the probability of early termination. The experiment reports

both the configured flow composition and the realised transaction composition, because a load comparison without this information confounds intensity with workload content.

Synthetic data also perform a methodological role. Metadata-driven generation creates structurally and semantically valid patient and document contexts without using personal data. Stateful orchestration associates every derived identifier, assertion, token, or document reference with one flow occurrence, while a session-aware identity hand-off prevents security material from crossing concurrent occurrences. The asynchronous execution model permits concurrency among independent occurrences while preserving sequential dependencies inside each occurrence. These architectural properties support controlled concurrency without weakening the semantics of the workflow.

The design includes an explicit calibration condition. A controlled local path establishes the range in which scheduling, event production, and analysis sustain the intended offered load without material drift. This calibration does not predict the capacity of a regional service. It determines whether the workload generator remains a credible experimental instrument. When delivery drift already increases in the control condition, a corresponding live scenario cannot support a conclusion about external saturation.

The statistical unit follows the same hierarchy. Requests within one flow share state, flows within one run share workload and environmental conditions, and repetitions provide the highest-level comparable units. Request distributions remain useful for transaction analysis, but the method does not treat every request as an independent replicate. This rule prevents a large sample count from overstating the amount of independent experimental evidence.

The selected strategy consequently favours internal validity over unsupported realism. It makes every assumption visible: flow weights, arrival profile, payload class, connection policy, seed, concurrency bound, execution mode, and outcome rule. An independent authorised traffic source can later test external validity without changing the measurement semantics. Until such a source ex-

ists, the thesis presents synthetic workloads as controlled experimental inputs and never as reconstructed regional demand.

2.2 Security-Constrained Observation Strategy

2.2.1 Authorised Observation Boundary

The Authorised Observation Boundary originates from regional security, privacy, and organisational control, not from a limitation of the workload generator. The Request Dataset Generator (RDG) can initiate approved synthetic traffic and retain sanitised client-side events. The experimenter cannot inspect private execution phases inside the Application Profile Management System (APMS), regional middleware, or downstream services, and cannot retain assertions, access tokens, credentials, certificate secrets, private endpoints, or personal data. The boundary therefore determines which causal statements the evidence permits[2].

This constraint leads to an *Advanced Black-box with Surrogate Observability* methodology. The black-box component treats the protected regional path as an externally observable service. The advanced component preserves more than a request timestamp: it observes planned and actual dispatch, transport and application outcomes, dependency propagation, terminal flow state, flow completion time, scheduling drift, concurrency, and matched path differentials. The surrogate component uses these correlated external signals to characterise the path without pretending to expose its internal phases.

The boundary changes the meaning of overhead. A mediated request can include authentication, authorisation, envelope handling, semantic validation, transformation, routing, queueing, upstream waiting, and backend processing. A client-side duration contains their composite effect together with network and client contributions. It does not equal gateway processing time, and a slower mediated response does not identify one internal component as its cause.

[2]DXC Technology, *Specifiche di integrazione autenticazione APMS*; Zanardini et al., *Infrastruttura di sicurezza GDL-O Sicurezza*.

Optional authorised internal timestamps can validate or refine the external estimate, but their absence limits causal attribution rather than invalidating the matched external comparison. Figure 2.1 expresses this methodological boundary. The internal functions remain analytically plausible contributors, but the experiment measures only their aggregate path effect.

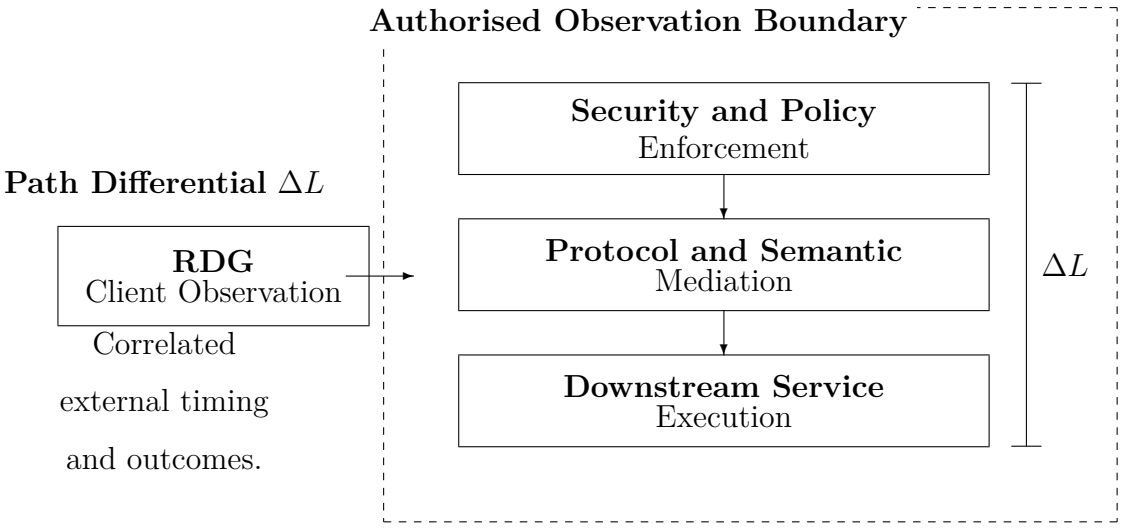


Figure 2.1: Security-constrained surrogate observability. The visual defines the limit of causal attribution adopted by the thesis. Source: Author's elaboration based on DXC Technology (2024) and Zanardini et al. (2024).

Security state remains part of the workload even when its value remains absent from the evidence. Reusing one assertion or token across unrelated occurrences can increase apparent throughput while violating the transaction contract and removing a mandatory acquisition cost. The method therefore scopes security outputs to one occurrence and records only whether their production and propagation succeed. Data minimisation removes protected values, not the causal position of the security transition.

Mock middleware events serve a separate purpose. They validate parsing, correlation, missing-field handling, and metric computation. They do not support claims about production routing, policy enforcement, queueing, or backend processing. This evidence separation prevents a test of the analysis chain from

becoming surrogate evidence about the regional infrastructure.

2.2.2 Client-Boundary Latency and Saturation Evidence

Iorio, Risso, and Casetti justify the choice of the client boundary by distinguishing network round-trip time from application-level flow completion time. Their end-to-end interval begins with request transmission and ends with complete response reception; it therefore captures transmission, transport, middlebox, application-protocol, and service contributions that a network probe does not expose[3]. This boundary also avoids the clock-synchronisation requirement of one-way measurements. For a protected regional path, the same principle makes the client the only common observation point across direct and mediated scenarios.

The full-stack interpretation makes Flow Completion Time (FCT) the principal end-to-end timing metric. Iorio et al. show that network round-trip time (RTT) captures only one layer of a latency budget that also contains transmission, forwarding, queueing, transport establishment, application protocol, security, service exposure, orchestration, and computation. RDG extends this observation boundary from one request-response exchange to the terminal outcome of an information-dependent workflow. Request latency remains necessary for locating which transaction population changes, but workflow FCT determines when the domain operation actually completes. This extension transfers the measurement principle, not the numerical results of the edge-computing campaign[4].

Figure 2.2 makes the measurement hierarchy explicit. The horizontal displacement between the network, transport, and application distributions represents progressively accumulated work rather than three interchangeable latency measures. The curves provide a conceptual reading of the hierarchy analysed by

[3]Marco Iorio, Fulvio Risso, and Claudio Casetti. "When Latency Matters: Measurements and Lessons Learned". In: *ACM SIGCOMM Computer Communication Review* 51.4 (Oct. 2021), pp. 2–13. DOI: 10.1145/3503954.3503956.

[4]Ibid.

Iorio et al.

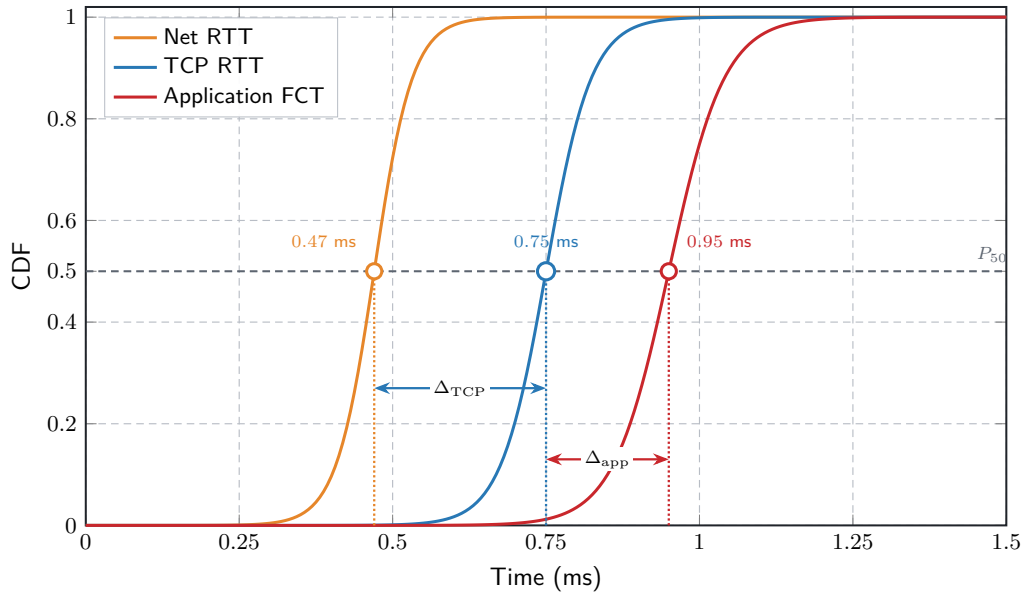


Figure 2.2: Conceptual separation of Network Round-Trip Time, Transmission Control Protocol, and application-level Flow Completion Time. Source: Adapted from Iorio et al. (2021).

For completed request i, the external latency is

$$L_i = t_i^{\text{response}} - t_i^{\text{dispatch}}. \tag{2.3}$$

For matched direct and mediated populations, the primary path estimand is

$$\Delta L = \widetilde{L}_{\text{mediated}} - \widetilde{L}_{\text{direct}}, \tag{2.4}$$

where $\widetilde{L}$ denotes a declared statistic over compatible transaction, payload, outcome, connection-policy, load, and time-window classes. The differential estimates the external effect associated with the path change. It does not decompose the contribution of APMS, transport, protocol transformation, or a downstream service.

Consequently, ΔL represents a *Path-level Overhead*: an aggregated external differential that can contain propagation, transport and security establishment,

gateway policy enforcement, SOAP/XML or REST/JSON handling, semantic validation, transformation, routing, queueing, and downstream service time. The term *aggregated* is essential. A positive differential supports the claim that the mediated path adds externally observable time under matched conditions; it does not identify which protected internal phase causes that difference. This interpretation follows the multi-layer latency budget in the edge-computing literature while respecting the narrower authorised observation boundary of the regional case.

The client boundary becomes informative about saturation only when latency joins other signals. Akram et al. model distributed load sharing as a race between communication delay and a stochastic availability window. Their analysis links increasing traffic intensity and communication latency to stale state, failed transfers, wasted work, and reduced effective throughput[5]. The load-sharing model does not become a model of the Veneto architecture, and its numerical thresholds do not transfer to RVE transactions. Its methodological contribution is the joint interpretation of latency, state validity, and completed useful work: delay becomes a saturation signal when it coexists with shrinking completion capacity and increasing ineffective activity.

Figures 2.3 and 2.4 translate this contribution into two complementary views. The state model distinguishes useful transitions from a transfer that arrives after the target state loses validity. The distribution view shows the same risk as a narrowing Load-Sharing Window (LSW) when traffic intensity increases. Both views remain methodological abstractions and import neither the paper's numerical thresholds nor a topology for the regional system.

[5]Saira Akram, Muhammad Asim Akram, and Fattah UR Rehman. "Numerical Evaluation of Network Latency and Throughput in Distributed Systems". In: *The Science Post* 1.4 (Dec. 2025). DOI: 10.5281/zenodo.18012513.

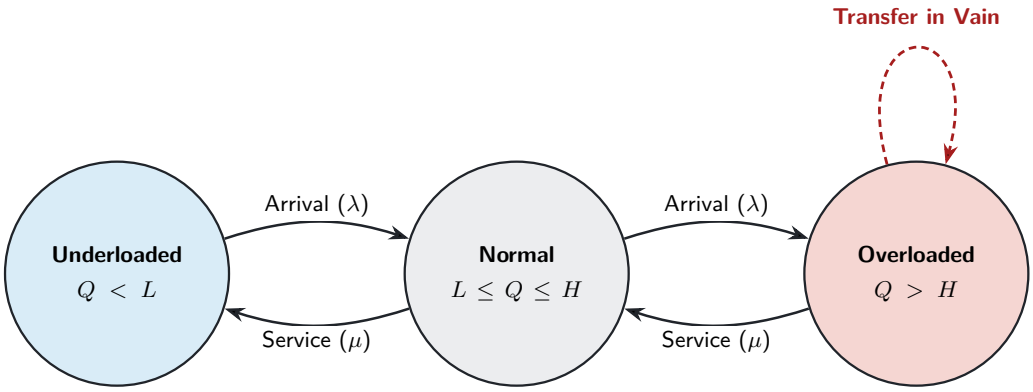


Figure 2.3: Conceptual transitions between underloaded, normal, and overloaded node states. Source: Adapted from Akram et al. (2025).

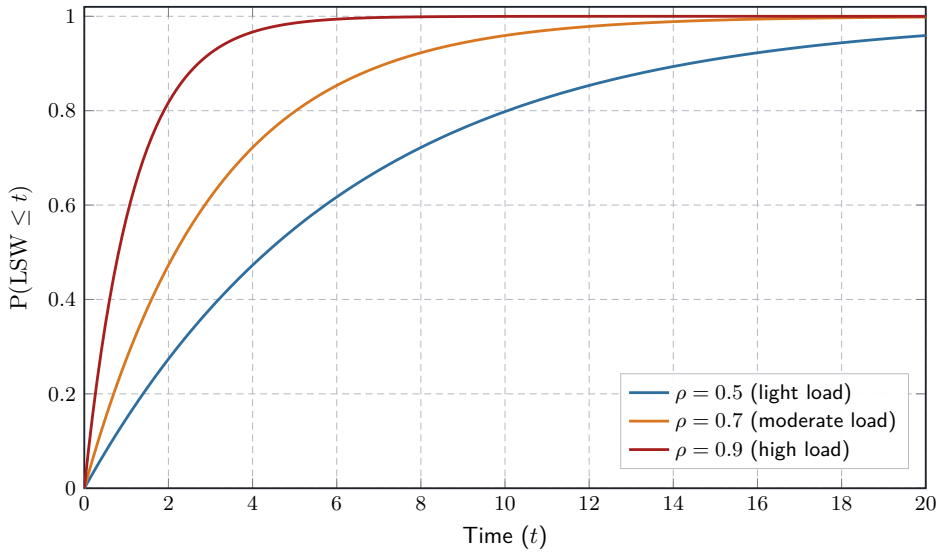


Figure 2.4: Conceptual cumulative distribution of the Load-Sharing Window under increasing traffic intensity. Source: Adapted from Akram et al. (2025).

This stochastic interpretation determines the summary statistics. The median (P_{50}) supplies a robust centre for a skewed latency population, while P_{95} and P_{99} expose the tail in which a delayed prerequisite is most likely to outlive the state that makes it useful. The arithmetic mean remains a supplementary descriptor, but it cannot reveal the probability of crossing a high-latency boundary and it changes markedly under rare delays. Akram et al. justify this distribution-aware choice through a quantile rule rather than through average

latency alone; Iorio et al. likewise evaluate application FCT at high percentiles. RDG therefore reports P_{50}, P_{95}, and P_{99} for a declared homogeneous population and observation window, together with sample size and outcome coverage. The Load-Sharing Window does not represent an actual SI.Ter component: it supplies a conceptual model for the stochastic interval in which information about distributed state remains actionable[6].

Dimitrov and Skjellum reinforce the need for application-level interpretation. Their experiments show that lower point-to-point latency does not necessarily produce lower application execution time, because communication pattern, message size, completion mode, CPU overhead, and the possibility of overlapping work determine the complete effect[7]. The studied Message Passing Interface (MPI) environment differs from healthcare interoperability, so the thesis transfers only the evaluation principle: a local latency number cannot stand for the performance of a heterogeneous workflow. Healthcare applications make this principle especially relevant because patient queries, context transitions, document retrieval, and cryptographic signing expose different message sizes, synchronisation points, and blocking dependencies.

An isolated ping-pong benchmark therefore answers a narrower question than the one posed by regional mediation. Dimitrov and Skjellum show that a lower point-to-point latency can coexist with higher processor overhead or reduced opportunity to overlap communication and computation; the regular request-reply pattern also omits the collective and asynchronous structures of real applications. In the healthcare case, an isolated endpoint call omits semantic validation, identity acquisition, branch selection, and failure propagation. RDG is consequently necessary as an orchestrator of dependent workflows, be-

[6] Akram, Akram, and Rehman, "Numerical Evaluation of Network Latency and Throughput in Distributed Systems"; Iorio, Risso, and Casetti, "When Latency Matters: Measurements and Lessons Learned".

[7] Rossen Dimitrov and Anthony Skjellum. *Impact of Latency on Applications' Performance*. Technical Report. Systems Research Group, Northwestern University, Apr. 2001. URL: https://users.cs.northwestern.edu/~srg/Papers/01-10-02/srg_2.pdf (visited on 07/31/2026).

cause the critical path exists only when each valid output enables the next step.[8].

Figure 2.5 provides an explanatory adaptation of the point-to-point and collective categories in their communication-pattern analysis. The original figures report message counts and sizes rather than sequence traces; the visualisation therefore illustrates the structural distinction without claiming to reconstruct an observed execution.

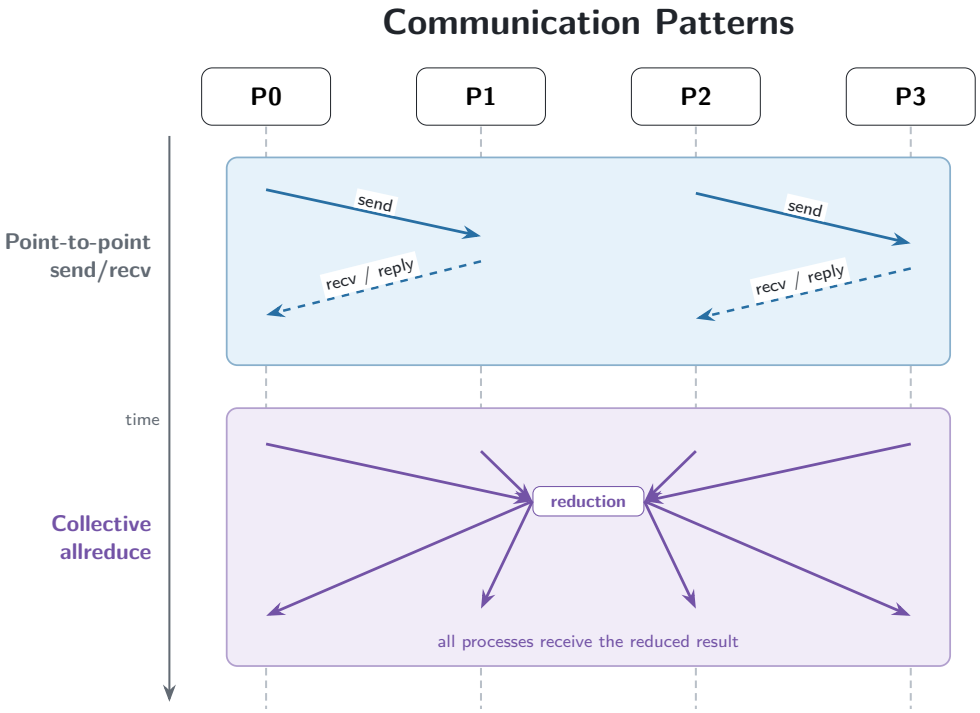


Figure 2.5: Conceptual point-to-point send/receive and collective all-reduce communication patterns. Source: Adapted from Dimitrov and Skjellum (2001).

Latency variability matters for the same reason. A median describes the centre of one population, but a dependent flow reacts to the slowest prerequisite on its critical path. Iorio et al. document dispersion, high-percentile instability, and latency spikes across access, transport, and application conditions; Dimitrov and Skjellum show that the application effect depends on the sur-

[8]Dimitrov and Skjellum, *Impact of Latency on Applications' Performance*.

rounding communication pattern. In a healthcare workflow, variability can delay the availability of a patient identity, context token, document reference, or signed artefact even when the median request remains stable. The method therefore reports distributions and tails for homogeneous populations and never substitutes a point-to-point mean for complete-flow behaviour.

Saturation is consequently an inference from converging evidence rather than a synonym for high latency. The method requires persistent scheduling drift, occupancy near the configured concurrency bound, a plateau or decline in useful throughput, and a corresponding change in latency, timeouts, or semantic failures. Stable response latency can coexist with increasing delivery drift when the generator no longer dispatches work on time. Conversely, higher latency without a throughput plateau can reflect payload composition, connection establishment, or remote environmental variation rather than saturation.

Table 2.2 fixes the operational meaning of the principal signals.

Table 2.2: Operational measures and their role in saturation analysis. Source: Author's synthesis based on Iorio et al. (2021) and Akram et al. (2025).

Measure	**Population and unit**	**Interpretive role**
Request latency	Homogeneous completed attempts; milliseconds.	Client-observed dispatch-to-response distribution for one transaction class, reported through P_{50}, P_{95}, and P_{99} with the sample size.
Flow completion	Complete and interrupted occurrences remain separate; milliseconds.	Critical-path effect of orchestration, waiting, and blocking failures.
Useful throughput	Successful terminal flows per declared window.	Completed domain work; it remains distinct from raw request activity.
Request throughput	Completed attempts per declared window.	Technical activity, including calls that belong to unsuccessful flows.
Scheduling drift	Actual minus planned start; milliseconds.	Delivery quality of the generator and early evidence of backpressure.
Outcome distribution	Transport, application, timeout, and terminal-flow classes.	Reliability and preservation of unsuccessful work in every denominator.

The literature therefore supports the measurement boundary and the multi-indicator reasoning, not an imported acceptance threshold. Client-boundary latency supplies a reproducible external signal; ΔL controls the matched path comparison; throughput, drift, concurrency, and outcomes determine whether the signal is consistent with saturation. Internal attribution remains outside the authorised boundary unless a new evidence source enters the protocol.

2.3 Healthcare Workflows as Information Dependencies

2.3.1 Protected Discharge and Anagrafe Zero

Protected discharge gives clinical-process meaning to the profiling strategy. The inpatient unit identifies the patient, prepares assessment information, proposes a destination setting, and forwards the request to the Centrale Operativa Territoriale (COT). The COT evaluates the request and assigns the definitive setting[9][10]. Each stage consumes information that a preceding stage makes available. The workflow is therefore a chain of semantic prerequisites, not a bag of independent service calls.

Patient identity illustrates the first dependency. The project documentation associates protected discharge with patient identification through Anagrafe Zero, while the Anagrafe Zero specification requires [RVE-54] Patient Query before other registry transactions[11]. A successful network response alone does not satisfy this prerequisite. The query has to produce an application outcome and a patient context that the next branch can use. Depending on the profiled outcome, a later operation can assign a new regional identity through [RVE-55] or update an existing identity through [RVE-57]. The branch follows the semantic result of the query; it does not follow only the HTTP status.

This dependency explains why a session-aware orchestrator is necessary. The patient context belongs to one synthetic occurrence and cannot leak into another concurrent occurrence. A failed query blocks the dependent branch, and a warning or application error remains distinguishable from transport success. A generic request generator can reproduce this behaviour only after the test author adds domain-specific state, extraction, validation, and branch rules. The profiling problem therefore resides in semantic continuity, not in the abil-

[9] Raggruppamento Temporaneo di Imprese guidato da AlmavivA, *Sistema Informativo Territoriale per gli Enti Sanitari della Regione del Veneto – Lotto 3*, pp. 18–21.

[10] Azienda Zero, *Realizzazione di un Sistema Informativo Territoriale per gli Enti Sanitari della Regione Veneto*, pp. 35–36.

[11] Vio et al., *Specifiche tecniche – Anagrafe Zero*.

ity to send a HL7-FHIR R4 request. Semantic FHIR R4 generation is therefore a condition of construct validity, not a convenience for producing payload volume. Anagrafe Zero interprets identifiers, demographic search parameters, resource structure, and transaction outcomes according to its declared FHIR contract. A syntactically valid but semantically arbitrary JSON or XML document can exercise transport while triggering a different validation branch from the represented patient operation. Metadata-driven generation preserves the profile-relevant structure and makes the resulting query, assignment, or update comparable across repetitions. It also keeps the data synthetic, so semantic fidelity does not require personal records[12].

The same principle applies to regional security and context access. An [RVE-1.b] interaction produces a SAML assertion; [RVE-121] uses the assertion and contextual information to produce the JWT required by [RVE-130][13]. The later call remains invalid when the required transition is absent, even if an earlier endpoint returns a nominally successful transport status. Occurrence-local state preserves both the security contract and the cost of acquiring that state.

Figure 2.6 makes the occurrence-local dependencies visible. It keeps the context path and the patient-query path separate: [RVE-130] consumes the material produced through [RVE-121], whereas [RVE-54] consumes the assertion in a distinct FHIR query. This separation prevents the visual model from suggesting one unsupported SAML–JWT–FHIR transaction chain.

[12] Vio et al., *Specifiche tecniche – Anagrafe Zero*.

[13] Zanardini et al., *Infrastruttura di sicurezza GDL-O Sicurezza*; Canal and Pavan, *Specifiche tecniche chiamata di contesto RVE*.

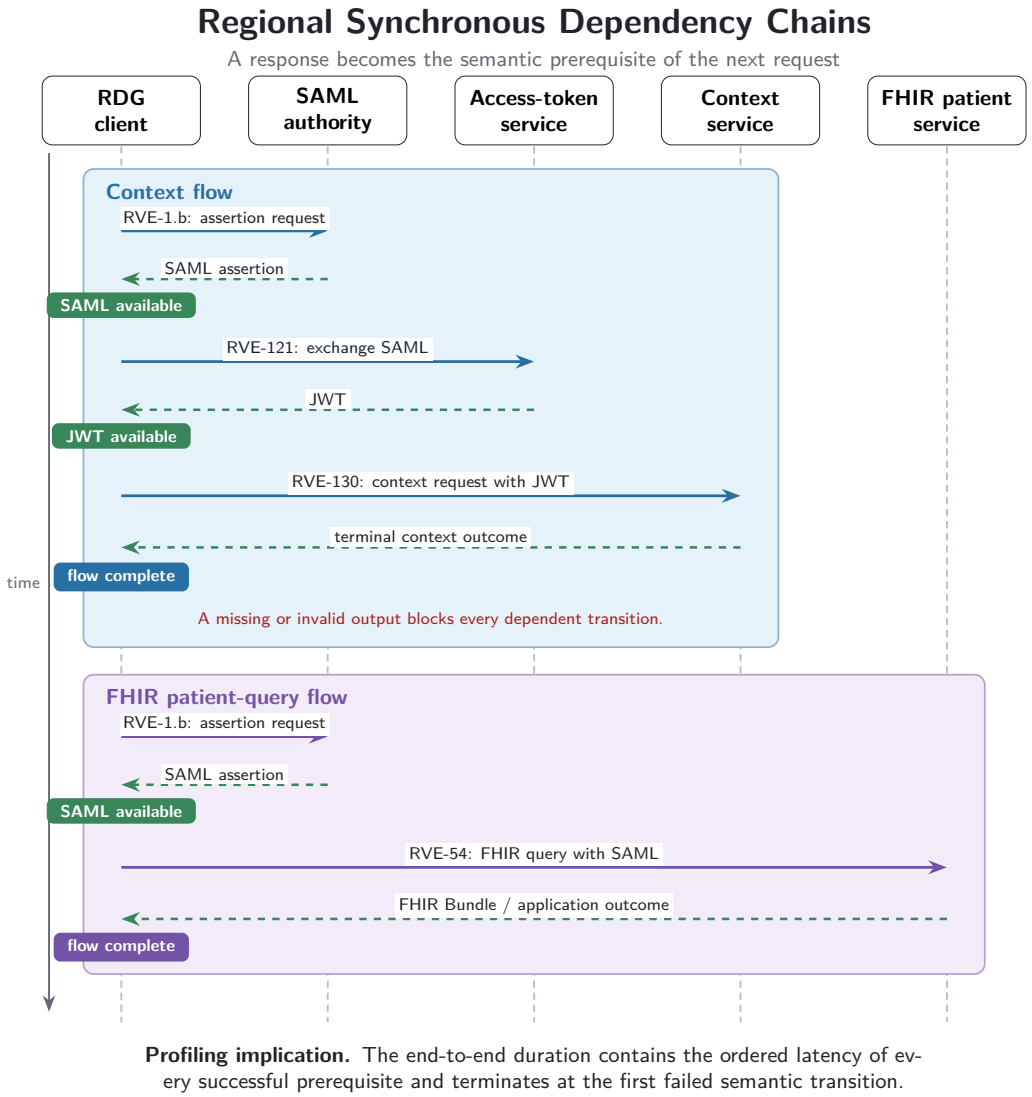


Figure 2.6: Conceptual regional dependency chains retained by the workload model. Source: Author's elaboration based on the regional security specification v2.14 (2024), context-call specification v1.3 (2025), and Anagrafe Zero specification v2.6 (2024).

Document access adds a further dependency family. Integrating the Healthcare Enterprise (IHE) Cross-Enterprise Document Sharing (XDS.b) separates metadata query, document selection, retrieval, and publication[14]. An Information

[14]IHE International, *IHE IT Infrastructure Technical Framework, Volume 1: Cross-Enterprise Document Sharing (XDS.b), Revision 20.2*; CNR (ICAR) e Dipartimento per la Trasformazione Digitale, *Specifiche tecniche per l'interoperabilità tra i sistemi regionali di FSE: Affinity Domain Italia*.

Technology Infrastructure ITI-18 query returns metadata; an ITI-43 retrieval consumes a repository and document reference; an ITI-41 publication carries document content and metadata. These transactions remain separate performance populations because they differ in payload, outcome, and position in the information chain.

Table 2.3 summarises the dependencies that shape the workload model.

Table 2.3: Information dependencies retained by the profiling model. Source: Author's synthesis of technical specifications.

Dependency	**Semantic prerequisite**	**Profiling consequence**
Patient identity	[RVE-54] produces a usable registry outcome before assignment or update.	The flow branches on application semantics and keeps later steps blocked after failure.
Security identity	[RVE-1.b] produces occurrence-local assertion evidence.	Protected calls include acquisition cost without retaining the assertion value.
Context session	[RVE-121] produces the material required by [RVE-130].	Transport success without the required output remains a failed state transition.
Document access	ITI-18 produces the reference consumed by ITI-43.	Query and retrieval retain separate latency, payload, and outcome populations.
Care transition	Patient identity and assessment precede COT evaluation and destination assignment.	Technical completion does not imply clinical approval or completion of the care pathway.
Document Signing	The SignOneDoc transaction needs a SAML Assertion Token from [RVE-1.b].	Transport success without the required output remains a failed state transition.

Both care transitions act as process frames. They identify the initiating actor, decision point, information prerequisites, and terminal organisational outcome. They do not justify an invented chain among every available RVE and ITI transaction. A transaction enters an executable case only when both a technical specification and a process source support its role. This source gate protects

the profiling model from turning architectural proximity into an unsupported workflow dependency.

Anagrafe Zero also demonstrates why throughput requires a domain denominator. Three successful endpoint responses do not necessarily represent three completed patient-identity processes. A query can end the flow, enable an assignment, or enable an update; warnings and duplicate-management rules can change the terminal meaning. Request throughput therefore measures technical activity, while flow throughput measures useful completion under an explicit semantic outcome rule.

2.3.2 Cryptographic and Protocol Heterogeneity

Scryba Sign supplies a controlled example of heterogeneous interoperability. The service surface uses SOAP, XML, and WSDL contracts over protected transport. The synchronous path combines identity and signer discovery through *GetUserInfo4* with the *SignOneDoc* operation. The document can travel as a Base64-encoded payload, and the service performs cryptographic work and constructs a signed artefact[15]. Scryba Sign therefore represents a heterogeneous interoperability constraint, not an optional software module. Treating it as an ordinary document upload removes the protocol, identity, and cryptographic work that the profile has to observe.

The methodological question is why the signing path enters the workload, not how the client constructs a particular envelope. A realistic profile has to retain the payload class, signing mode, session semantics, and terminal outcome because each factor changes the critical path. SOAP/XML handling adds representation and envelope costs; Base64 increases the transferred representation relative to the binary source; protected transport adds connection and security work; cryptographic signing adds service-side computation. The Authorised Observation Boundary makes only their aggregate external effect observable as part of the Path-level Overhead. Signer identity creates a further dependency.

[15]Medas S.r.l., *Scryba Sign 3.x Developer's Guide*, pp. 17–18, 91–96, 107–111.

The information-discovery step identifies the available signing powers, certificates, and One-Time Password (OTP) devices; the signing request depends on an admissible certificate and on the applicable OTP authorisation path[16]. This relationship requires session-aware identity hand-off: certificate selection and One-Time Password state belong to the same signing occurrence and a failed identity prerequisite blocks the cryptographic operation. The complete FCT consequently includes the ordered identity and signing path rather than only the final SOAP response. The current executable perimeter preserves the order *GetUserInfo4–SignOneDoc* for the synchronous population; automated certificate selection and the complete OTP lifecycle remain contract-level extensions, so the thesis does not present them as executed measurements.

The synchronous and asynchronous scenarios define distinct experimental populations. In the synchronous scenario, the request-response chain includes the return of the signed document. In the asynchronous scenario, submission produces session and document identifiers, while later status checks and retrieval define completion. The asynchronous path therefore introduces queue residence, polling or notification policy, and a longer state lifecycle[17]. Pooling the two modes in one latency distribution destroys construct validity because their terminal events and critical paths differ.

The profiling model consequently stratifies signing evidence by execution mode, payload class, connection policy, and application outcome. The current executable evidence perimeter includes the synchronous signing population. The asynchronous contract remains a separately defined profile and does not acquire results by analogy. This boundary preserves architectural completeness without presenting an unexecuted scenario as observed evidence.

[16] Medas S.r.l., *Scryba Sign 3.x Developer's Guide*, pp. 60–62, 91–96.

[17] Ibid., pp. 17–18.

Table 2.4: Methodological separation of Scryba Sign profiling populations; the distinct terminal conditions prevent synchronous, asynchronous, and large-document paths from being pooled into one latency distribution. Source: Author's synthesis based on the Scryba Sign 3.x Developer's Guide v16 (Medas S.r.l., 2024).

Profile	**Terminal condition**	**Relevant overhead**
Synchronous signing	The signed artefact returns in the synchronous interaction.	SOAP/XML and Base64 transfer, protected transport, cryptographic processing, and response transfer.
Asynchronous signing	Submission state leads to later availability and retrieval of the signed artefact.	Submission, queue residence, status acquisition, retrieval policy, and state lifecycle.
Large-document path	The signed artefact completes through the applicable document-transfer procedure.	Document size, hashing, external transfer, signing, and retrieval remain explicit strata.

A direct-versus-mediated comparison uses the same signing mode, payload class, application outcome, and connection policy on both paths. Under these controls, ΔL estimates the composite external effect associated with mediation. It does not isolate cryptographic time, SOAP serialisation, policy enforcement, or network transfer. The signing service therefore strengthens the need for surrogate observability: it creates a relevant, heterogeneous critical path while leaving its protected internal decomposition unavailable.

The scenario also confirms the importance of complete-flow metrics. A fast submission does not imply a fast signed-document outcome, just as a successful patient query does not imply completion of a protected-discharge process. The method keeps request latency, state-transition success, and terminal flow

completion distinct so that local progress cannot masquerade as domain-level completion.

2.4 Experimental Design and Evidence Semantics

2.4.1 Controls, Variables, and Reproducibility

The primary estimand is the change that RDG observes when a documented operation follows a mediated path instead of a direct path under matched conditions. The independent variable is the selected path. Transaction class, payload class, semantic outcome rule, flow position, offered-load profile, connection policy, concurrency bound, seed policy, duration, and observation window act as controlled variables. Request latency, flow completion, useful throughput, scheduling drift, and outcome distribution act as dependent variables.

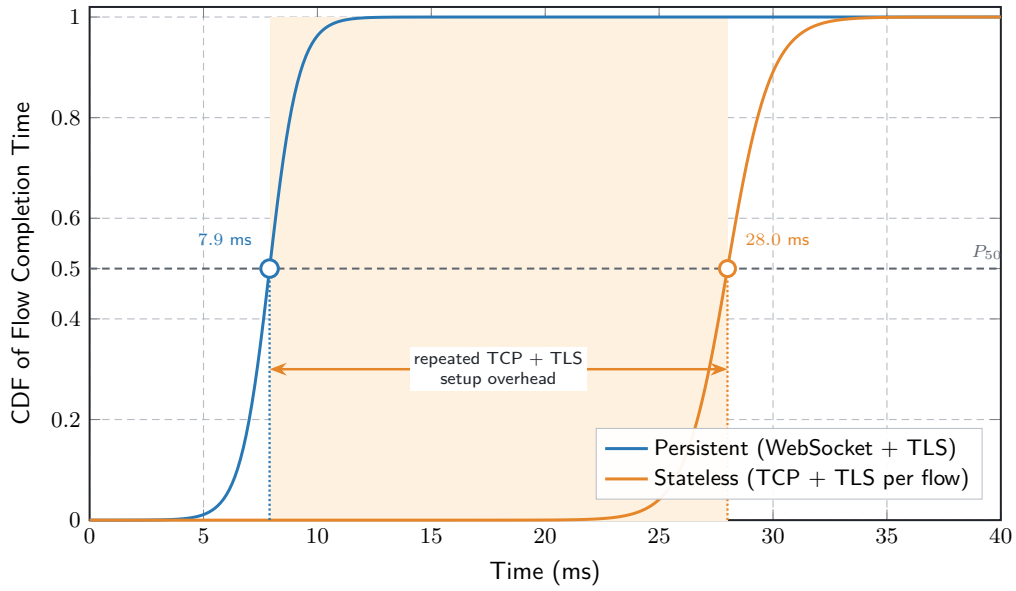
Matching requires more than a shared transaction identifier. Direct and mediated populations use compatible payloads, success definitions, flow positions, and workload conditions. Scenario order also remains controlled because connection reuse, cache state, or remote environmental drift can create an apparent path difference. Paired seeds align generated choices, while deterministic shuffling or interleaving reduces a fixed chronology effect. The method retains run timestamps and dispersion because a live remote environment remains variable even when the synthetic workload is reproducible.

Table 2.5 identifies the main confounding sources and the residual limitations that remain after control.

Table 2.5: Controls for an interpretable direct-versus-mediated differential. Source: Author's synthesis based on Zhu et al. (2005) and Iorio et al. (2021).

Confounding source	**Experimental control**	**Residual limitation**
Transaction semantics	Match operation, flow position, and application outcome.	Contract evolution can change equivalence across versions.
Payload and connection state	Stratify payload class and declare connection policy.	Private caches and remote connection management remain opaque.
Offered load	Reuse temporal profile, mix, seed policy, duration, and concurrency.	Unobserved external demand remains uncontrolled.
Execution chronology	Pair repetitions and control scenario order.	Environmental drift can persist within and across pairs.
Generator capacity	Calibrate delivery and retain drift and active-concurrency signals.	Host scheduling remains part of the external experiment.

Every run identifies the workload profile, offered rate, duration, seed, burst parameters, flow mix, concurrency bound, execution mode, configuration state, and artefact version. Reusing a seed aligns paired choices; changing seeds across repetitions samples alternative schedules. Neither action makes the remote environment deterministic. Reproducibility refers to the experimental input and analysis procedure, while environmental repeatability remains an empirical question. Connection policy deserves explicit control because security setup can dominate short flows. Figure 2.7 gives a conceptual reconstruction of the persistent and stateless populations analysed.



RDG design implication. Preserve occurrence-local security state and reuse transport sessions whenever the transaction contract and the authorised security policy permit it.

Figure 2.7: Conceptual comparison of Flow Completion Time. Source: Adapted from Iorio et al. (2021).

Durations use a monotonic clock. Wall-clock timestamps establish chronology but do not support subtraction across independent hosts without a documented synchronisation bound. Correlation requires compatible identifiers, event semantics, and populations. Missing or unmatched fields remain absent instead of being inferred. These rules prevent timestamp precision from being confused with visibility beyond the authorised boundary.

Structured events retain run, scenario, occurrence, flow, step, and request identity together with planned time, measured duration, and outcome class. Normalisation preserves malformed records, missing terminal events, interrupted flows, and unmatched correlations as data-quality outcomes. Silent removal of these records biases both latency and reliability toward successful observations. Provenance therefore precedes performance interpretation.

2.4.2 Evidence Gates and Ex Ante Judgement

Every performance statement requires an operational definition, a population, a unit, an aggregation level, and an event source. If $C(w)$ denotes the number of completed units in a window w, then

$$X(w) = \frac{C(w)}{|w|} \tag{2.5}$$

defines throughput only after the method declares whether $C(w)$ counts requests, terminal flows, or successful terminal flows. Retries can increase request throughput without increasing useful flow throughput, and early failures can reduce request activity while increasing the number of incomplete flows.
Criteria remain fixed before the corresponding results enter the judgement. For criterion j, the thesis uses

$$C_j = (S_j, P_j, m_j, s_j, w_j, d_j, \tau_j, E_j), \tag{2.6}$$

where S_j is the scenario, P_j the population, m_j the metric, s_j the statistic, w_j the window, d_j the preferred direction, τ_j an externally justified threshold, and E_j the evidence gate. A numerical value is not assessable when its population, boundary, or threshold provenance is missing.

Table 2.6: Ex ante criteria and required evidence. Source: Author's methodological synthesis based on the SI.Ter procurement specification (Azienda Zero, 2025).

Criterion	**Population**	**Required evidence**
Latency	Homogeneous requests and complete or interrupted flows remain separate.	Timer boundary, statistic, sample size, window, repetitions, path, and outcome rule.
Throughput	Request and useful-flow completions under declared offered load.	Profile, mix, concurrency, calibration, duration, and plateau rule.
Reliability	Transport, application, timeout, and terminal-flow outcomes.	Error taxonomy, denominator, retry rule, and semantic outcome rule.
Saturation	Drift, active concurrency, throughput, latency, and failures.	Repeated convergence of indicators rather than offered rate alone.
Data quality	Expected, parsed, malformed, incomplete, and unmatched events.	Event semantics, provenance, coverage rule, and completeness threshold.

Observation completeness conditions every statistic. The configured timeline and flow definition determine expected requests and terminal events; the retained event stream determines parsed observations. A missing end event, duplicate identifier, malformed record, or unmatched occurrence changes the admissible population. Successful-request latency therefore appears beside timeouts and application failures rather than replacing them.

The judgement vocabulary follows the evidence hierarchy. A criterion is *not assessable* when its threshold or evidence gate is absent; *conditionally supported* when structural or controlled evidence exists without the required live coverage; *accepted* when valid evidence meets an externally declared rule; and *rejected*

when valid evidence violates that rule. Mock evidence supports instrument validation. Live external evidence supports observations for the declared environment. Internal causal evidence requires a newly authorised source inside the protected boundary.

Threshold provenance remains external to the campaign under judgement. An approved service-level requirement, contractual criterion, or preregistered protocol can supply τ_j; an observed percentile cannot become its own pass value. The Sistema Informativo Territoriale (SI.Ter) procurement specification supplies a bounded front-end reference for interactions that do not depend on third-party systems[18]. That reference does not automatically transfer to mediated RVE workflows because their dependencies and observation boundary differ.

Repeated scenarios reduce but do not remove temporal confounding. Paired seeds align the synthetic choices, controlled ordering limits a fixed sequence effect, and run timestamps retain the chronology needed to detect environmental change. Remote observations within one run remain dependent. The final judgement therefore considers run-level dispersion and consistency of direction instead of relying only on a pooled request distribution.

Negative evidence remains part of the decision. A malformed event, missing terminal state, failed token transition, timeout, or generator overload can make a performance criterion not assessable while still exposing a framework or contract problem. Data-quality and semantic gates apply before aggregation, and their failures remain visible in the reported denominator.

2.5 Generic versus Domain-Specific Profiling

2.5.1 Modelling Distance and Semantic Fidelity

Apache JMeter and the Request Dataset Generator (RDG) represent different modelling paradigms rather than different protocol capabilities. JMeter pro-

[18] Azienda Zero, *Realizzazione di un Sistema Informativo Territoriale per gli Enti Sanitari della Regione Veneto*, pp. 56–57.

vides a mature request-centric model through samplers, controllers, extractors, variables, assertions, scripts, and distributed execution[19]. RDG uses a typed healthcare flow that integrates security, data, dependencies, scheduling, and evidence semantics.

Both paradigms issue concurrent SOAP/XML and REST/JSON requests. The decisive criterion is *modelling distance*: the additional interpretation and state logic required to express a documented regional workflow. A generic plan coordinates variables, extractors, assertions, controllers, and scripts to propagate security material, interpret patient-query outcomes, preserve context and document references, and block invalid transitions. RDG encodes these relations in the workflow vocabulary. The generic paradigm remains capable, but semantic fidelity depends on author-maintained conventions distributed across plan elements. This distribution increases the risk of cross-user state leakage, execution after transport-only success, or exclusion of failed prerequisites from the useful-flow denominator. Encoding these relations as workflow invariants reduces those failure points.

RDG also couples workload and evidence construction. Domain metadata creates synthetic patient and document contexts; stateful orchestration retains occurrence-local outputs and branches; asynchronous execution preserves concurrency among independent flows; and structured events distinguish technical attempts from terminal domain outcomes. The resulting model translates documented healthcare dependencies into executable, observable experimental units. Table 2.7 summarises the resulting trade-off.

[19] Apache Software Foundation. *Apache JMeter User's Manual: Elements of a Test Plan.* URL: https://jmeter.apache.org/usermanual/test_plan.html (visited on 08/04/2026); Apache Software Foundation. *Apache JMeter User's Manual: Component Reference.* URL: https://jmeter.apache.org/usermanual/component_reference.html (visited on 08/04/2026); Apache Software Foundation. *Apache JMeter User's Manual: Functions and Variables.* URL: https://jmeter.apache.org/usermanual/functions.html (visited on 08/04/2026); Apache Software Foundation. *Apache JMeter User's Manual: Remote (Distributed) Testing.* URL: https://jmeter.apache.org/usermanual/remote-test.html (visited on 08/04/2026).

Table 2.7: Generic and domain-specific profiling paradigms comparison. Source: Author's synthesis of the Apache JMeter User's Manual (Apache Software Foundation, accessed 2026) and the Anagrafe Zero v2.6, regional security v2.14, context-call v1.3, and Scryba Sign v16 specifications.

Dimension	**Generic request-centric paradigm**	**Domain-specific workflow paradigm**
Unit of analysis	Request or sampler sequence extended with user-defined state.	Typed information-dependent healthcare flow.
State semantics	Scope and lifetime emerge from variables, extractors, controllers, and custom logic.	Occurrence-local state and prerequisite blocking form explicit invariants.
Data semantics	Payloads remain generic unless the plan incorporates domain-specific generation and validation.	Synthetic patient and document metadata preserve the represented profile without using personal data.
Outcome semantics	Transport assertions require additional domain rules for application and flow completion.	Transport, application, and terminal-flow outcomes remain separate by construction.
Observability	Endpoint timing is native; workflow correlation and path differentials require explicit composition.	Correlated external events directly support surrogate path observation.
Evolution cost	Broad ecosystem and mature distributed facilities; domain contracts remain test-plan responsibilities.	Lower modelling distance for represented contracts; domain model requires maintenance when specifications change.

The comparison therefore concerns semantic risk and experimental traceability. A generic plan is attractive when the target is a stateless endpoint or a protocol-level baseline. A domain-specific workflow is attractive when the experimental outcome depends on ordered state transitions, domain metadata, application-level validation, and flow-level denominators. The Veneto use case belongs

to the second class because security, identity, context, document, and signing transitions determine whether useful work completes.

2.5.2 Justification of the Request Dataset Generator framework

RDG is methodologically justified when five conditions hold: the target process contains dependencies whose violation changes experimental meaning; synthetic data preserve contract structure without protected material; the research question concerns complete-flow behaviour and path differentials; and one evidence model connects offered load, state transitions, external timing, and terminal outcomes. A throughput comparison with JMeter requires matched workloads, equal host conditions, and a dedicated generator benchmark. A stateless generic baseline remains useful for calibration and protocol isolation, but it does not replace a session-aware workload for protected regional workflows. The choice relocates rather than eliminates complexity. A domain-specific model requires versioned transaction semantics, outcome definitions, scheduling controls, and event provenance. The same elements otherwise reappear as dispersed custom logic in a generic plan; RDG makes them inspectable at the abstraction level of the research question.

2.5.3 Scientific Rationale for Domain-Specific Profiling

RDG responds to latency-aware distributed systems by combining quantile-based stochastic interpretation, full-stack FCT, matched Path-level Overhead, application-representative communication patterns, metadata-driven FHIR R4 generation, and session-aware orchestration. The absence of authorised traces makes controlled synthetic workflows the admissible internally valid input, while the regional security perimeter imposes Advanced Black-box with Surrogate Observability. RDG is therefore not merely custom software: it translates international methodological consensus into Regione Veneto's operational, semantic, and security constraints.

2.6 Summary

Within the available evidence perimeter, controlled Synthetic Workflows constitute the thesis's only experimentally valid instrument. Operational observation cannot isolate offered load from environmental variation, protocol benchmarks omit domain dependencies, and trace replay requires authorised traces with reconstructable state and ordering. Because the permitted corpus contains no such trace, synthetic execution provides reproducible inputs while leaving external representativeness unproven. It also makes flow composition, arrival timing, randomisation, payload classes, and outcome rules explicit experimental factors.

The regional authorisation perimeter determines the observation method. Advanced Black-box with Surrogate Observability treats the protected path as a black box and correlates dispatch timing with client-boundary latency, application outcomes, dependency propagation, and flow completion. Scheduling drift, concurrency, and direct-versus-mediated differentials provide path-level evidence without assigning delay to inaccessible internal phases. Mock events validate the measurement chain; live claims require authorised observations and complete provenance.

Semantic fidelity completes the method. Fast Healthcare Interoperability Resources Release 4 (FHIR R4) payloads preserve profile-relevant meaning, while stateful orchestration keeps assertions, tokens, identifiers, document references, and signing state local to each flow occurrence. Prerequisites remain sequential within workflows and independent occurrences execute concurrently, so performance populations represent completed or interrupted healthcare operations rather than unrelated HTTP exchanges. Together, synthetic control, surrogate observability, and semantic fidelity define credible profiling requirements. Chapter 3 shows how the Request Dataset Generator turns these principles into an executable profiling instrument.

Chapter 3

Selected Approach

This chapter presents the Request Dataset Generator as both the selected engineering approach and the experimental instrument that produces the thesis's empirical evidence. It first specifies the architecture, synthetic inputs, execution model, and evidence pipeline; it then defines the evaluation protocol and quality controls; finally, it characterises the resulting empirical stochastic distributions, assesses quantile-based confidence, and interprets the Path-level Overhead (ΔL) under the surrogate observability invariants established in Chapter 2.

Chapter 1 defines the need for a controlled and traceable evaluation of healthcare-interoperability workflows, while Chapter 2 derives the methodological requirements imposed by that need. The implementation described here is the Request Dataset Generator (RDG) repository. The analysis distinguishes three evidential levels. A feature is *implemented* when it is present in the current code; it is *functionally exercised* when a test or archived run covers it; and it is *experimentally evaluated* only when an identified configuration, execution log, normalised dataset, and metric report support the corresponding analysis. In particular, mock execution validates the framework pipeline but does not measure the capacity of a live regional or industrial service.

3.1 Technical Objectives and System Architecture

3.1.1 Architectural Objectives and End-to-End Pipeline

To translate the high-level evaluation requirements into an executable platform, the Request Dataset Generator relies on a modular architecture divided into clear functional boundaries. This subsection defines the core technical objectives of the framework and describes the end-to-end processing pipeline that transforms raw input configurations into structured performance evidence.

Technical objectives

The framework translates the methodological requirements of Section 2.2.2 into five technical objectives. First, it must generate synthetic healthcare data without requiring personal data. Second, it must compose those data into ordered multi-step flows whose dependencies remain explicit. Third, it must schedule independent flow occurrences according to a configurable temporal model while preserving the order of steps within each occurrence. Fourth, it must emit structured evidence at request, flow, and time-window granularity. Finally, it must make configuration, random seeds, execution mode, and output provenance recoverable from each run.

These objectives delimit the artefact. RDG is not a conformance-certification tool, a production audit system, or a clinical simulator. It creates and executes controlled workloads at the client boundary and can enrich the resulting records when a compatible middleware log is supplied. The absence of that second source remains visible in the normalised dataset rather than being replaced by estimated values.

End-to-end pipeline

The command dispatcher in `main.py` exposes independent commands for patient generation, flow-dataset construction, timeline production, execution, log collection, and metric calculation. The `pipeline` command connects all the stages in a single invocation:

$$\begin{aligned}\text{configuration} &\rightarrow \text{synthetic data} \rightarrow \text{timeline} \\ &\rightarrow \text{execution log} \rightarrow \text{normalised dataset} \rightarrow \text{metrics and reports.}\end{aligned}$$

This separation allows an intermediate artefact to be inspected or replayed without changing the responsibilities of the surrounding stages.

Design Patterns

RDG adopts a staged Pipeline separating generation, scheduling, execution, normalisation, and analysis. The handler map in `main.py` forms a lightweight Command dispatcher. Transaction modules act as Builders of uniform steps, which `FlowOrchestrator` composes according to the `RveTransactionModule` protocol. Traffic-profile functions implement Strategy, whereas `MtlsContext Factory` applies Factory with lazy caching to mTLS contexts. The configuration Bridge isolates graphical-interface persistence from the core.

Table 3.1: Each pipeline stage owns one transformation and emits an inspectable artefact. Source: Author's elaboration based on the framework's source code.

Stage	**Implementation**	**Produced evidence**
Data generation	`fhir_patient_ generator.py`	Synthetic Patient resources and FHIR Bundles.
Workload composition	`request_dataset_ generator.py`, `flow_ orchestrator.py`	Weighted population of flows, ordered steps, identifiers, dependencies, and payload metadata.
Temporal scheduling	`timeline_ generator.py`	Timestamped flow occurrences with base/burst provenance and experiment identifiers.
Execution	`traffic_exec_ engine.py`	Mock or live step outcomes and JSONL request/response/flow events.
Collection	`log_collector.py`	Request, flow, and bucketed time-series populations; optional middleware enrichment.
Analysis	`metric_engine.py`	Machine-readable metrics, human-readable report, comma-separated-value tables, warnings, and optional graphs.

3.1.2 Module Boundaries and Secure Transport

Repository organisation and dependency direction

The top-level modules implement the pipeline stages, while the transaction package contains the transaction builders and the flow orchestrator. A transaction builder returns one executable step with its method, target selected from configuration, headers, body, transport profile, dependency specification, output-extraction rules, and possibly a synthetic response (generated only when in mock mode). The orchestrator adds correlation metadata and combines these steps into flow types. The execution engine consumes this common shape; it does not contain healthcare-specific request templates.

This direction of dependency is important. Domain-specific contracts remain in the transaction modules, composition remains in the orchestrator, and temporal or analytical mechanisms operate on shared metadata. Adding a new transaction therefore does not require the timeline generator or metric engine to understand its payload, provided that the module satisfies the common step contract and declares the outputs required by downstream steps.

Configuration and run provenance

Configuration is obtained by merging a global default configuration with a scenario-specific JavaScript Object Notation (JSON) file. Before a pipeline execution, the dispatcher establishes a scenario identifier, run identifier, execution identifier, and seeds for workload, timeline, and fault injection. It also resolves output paths under a run-specific directory and writes the effective configuration as `config.resolved.json`. The run directory can then contain the generated dataset, timeline, JSONL execution log, normalised metrics, metric reports, tabular summaries, and graphs.

The effective configuration is more authoritative than the default template when interpreting an archived run: templates may evolve after an execution, whereas the resolved copy records the parameters attached to that run. Cre-

dentials, security tokens, certificate material, private endpoints, and generated clinical-like values are deliberately excluded from this chapter.

Mutual Transport Layer Security context management

`MtlsContextFactory` separates cryptographic material from executable steps. A transaction declares only a named mutual-Transport Layer Security (mTLS) profile in its transport metadata; the runtime configuration retains certificate paths, verification policy, and any environment-variable reference used to resolve a certificate password. On first use, the factory constructs a Python Secure Sockets Layer (*SSLContext*) object, loads either a Privacy-Enhanced Mail (*PEM*) certificate chain or a Public-Key Cryptography Standards (*PKCS* #12) container, and caches the context by profile name. Subsequent requests reuse the same context through the run-wide `aiohttp.ClientSession`, allowing eligible secure connections to benefit from session-level connection pooling without copying keys or passwords into generated datasets.

The factory returns no context when a transport profile is absent or disabled and raises an explicit error when an enabled profile lacks its certificate. This behaviour makes transport policy observable at configuration level while keeping sensitive material outside request logs and thesis artefacts. It also preserves the experimental distinction between protocol semantics and secure channel establishment: a failed mTLS setup is a transport outcome, not an application response from the regional gateway.

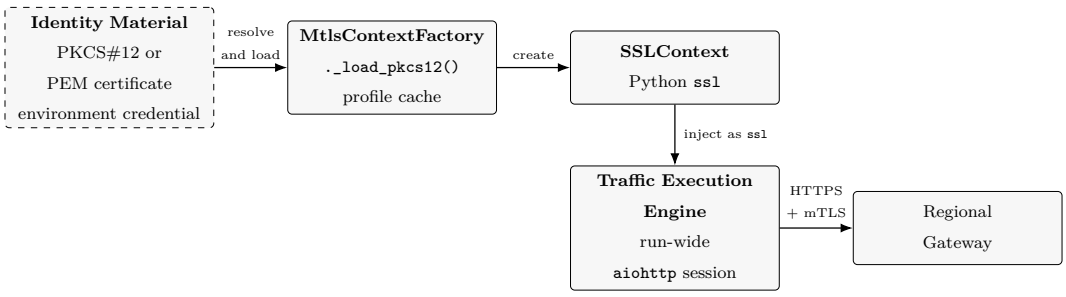


Figure 3.1: Secure Transport and mutual Transport Layer Security Management Layer. Source: Author's elaboration based on the framework's source code.

3.2 Synthetic Inputs and Executable Transactions

3.2.1 Synthetic Resources and Transaction Heterogeneity

The execution of stateful healthcare workflows requires generating rich synthetic payloads that conform to regional schemas while remaining completely decoupled from real patient records. This subsection details the demographic and clinical structures produced by the data generator and analyses the cryptographic and protocol heterogeneity that characterises the executable transactions of the campaign.

Synthetic Patient resources

The data generator creates a FHIR Patient-shaped resource with synthetic identifiers, demographics, contact information, addresses, regional extensions, a contained practitioner, and a contained assistance contract. It can also construct the transaction Bundle used by the [RVE-55] path and serialise generated structures as JSON or Extensible Markup Language (XML). Random generation removes the need for real patient records, while an optional identifier and a declared seed support controlled regeneration.

The generated structure is intended to exercise the contracts implemented by the framework. Its richness does not prove conformance with every cardinality, terminology rule, or business constraint of the applicable regional profile. That distinction mirrors the boundary established in Section 2.3.1: the code demonstrates what the workload can produce, while the primary specification remains authoritative for conformance.

Workload population and correlation model

`build_workload()` accepts the number of source flows, a weighted `flow_mix`, an execution mode, the complete configuration, the number of patients and an optional seed. Weights are normalised and sampled to choose a flow type; the number of synthetic patients is generated for each step of the source flow

that needs that specific resource. The resulting flow carries a flow identifier, a type, and an ordered list of steps. Every step has its own request identifier and index, while the timeline generator later adds a distinct `flow_occurrence_id` whenever a source flow is scheduled. This last identifier prevents repeated uses of the same source flow from being merged during log reconstruction.

The implementation defines the seven flow types in Table 3.2.

Table 3.2: Implemented integration-flow map, including each ordered transaction chain and its number of executable steps. Source: Author's elaboration based on the framework's source code and regional specifications.

Flow type	Ordered transaction modules	Steps
`FLOW_CONTEXT_CALL`	[RVE-1.b] → [RVE-121] → [RVE-130]	3
`FLOW_PATIENT_QUERY`	R[VE-1.b] → [RVE-54]	2
`FLOW_PATIENT_CREATE`	[RVE-1.b] → [RVE-55]	2
`FLOW_ITI_18`	[RVE-1.b] → ITI-18	2
`FLOW_ITI_43`	[RVE-1.b] → ITI-43	2
`FLOW_PATIENT_QUERY_MIDDLEWARE`	RVE token → gateway RVE-54 → gateway RVE-55 → gateway RVE-57 → gateway RVE-100	5
`FLOW_SCRYBASIGN_SIGN`	GetUserInfo → SignOneDoc	2

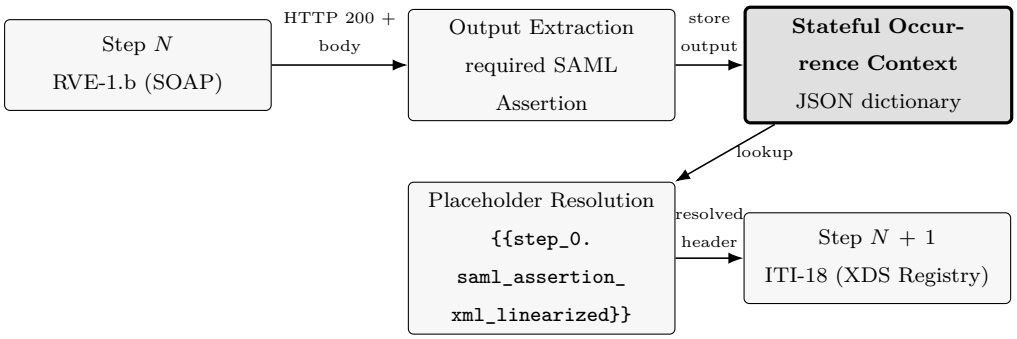


Figure 3.2: Semantic transformation and placeholder resolution across dependent protocol steps. Source: Author's elaboration based on the framework's source code.

Regione Veneto, IHE Transactions, and mediated paths

The direct regional paths combine an initial assertion-producing transaction with context, patient-registry, or document transactions. The two ITI flows use dedicated transaction modules after the initial integration step. The mediated patient path has a different structure: it obtains a middleware token and

then executes the configured gateway scenarios for [RVE-54], [RVE-55], [RVE-57], and [RVE-100]. This five-step path is therefore not a request-for-request replacement for the two-step direct patient query. Comparisons between their completion times must acknowledge the different step count and contracts.
The code confirms that [RVE-57] is present in the mediated flow, whereas the permitted source corpus contains no authoritative specification that establishes its complete domain semantics. The chapter therefore treats [RVE-57] as an implemented transaction label and excludes it from claims of semantic equivalence with the direct path described in Section 2.3.

Digital-signature provider flow

`FLOW_SCRYBASIGN_SIGN` composes two SOAP/POST steps in a fixed order: the provider user-information operation and the single-document signing operation. The shared `scrybasign_common.py` module centralises endpoint selection, mTLS-profile resolution, and construction of the HTTP `Authorization` header. In particular, `resolve_auth_header()` reads a scoped username and password, optionally resolves the password from an environment variable, encodes the credential pair in Base64, and returns the Basic-authentication value consumed by both builders. The generated dataset therefore contains the application header but not the certificate material managed by `MtlsContextFactory`.
The signing envelope carries its document in the Base64 `docBin` field. The current builder embeds a fixed test artefact and `resolve_auth_header()` implements transport-level Basic authentication.
The provider documentation distinguishes synchronous and asynchronous signing surfaces.[1] The current flow map instantiates only the synchronous path. The workload model could represent the two interaction styles as distinct flow types, step sequences, and completion criteria, but an asynchronous signing experiment requires explicit enqueue, status, or notification modules that the

[1] Medas S.r.l., *Scryba Sign 3.x Developer's Guide*.

source does not implement; Section 3.6 retains that evidential boundary.

Step dependencies and the mock/live boundary

A flow is more than a list of independent HTTP requests; a step may also declare that a field comes from an output of an earlier step. In live mode, downstream headers, bodies, and URLs can contain scoped placeholders; the execution engine resolves them from the occurrence-local context only after processing the preceding response. Required outputs are validated before the chain continues; an unresolved placeholder or missing required output is logged as a failure. In mock mode, transaction builders attach synthetic responses and the orchestrator propagates their declared outputs while constructing the chain. Execution then consumes those responses without opening an HTTP session. This mode tests composition, scheduling, failure handling, logging, collection, and metric production under controlled conditions. It cannot measure network, gateway, provider, or production-system behaviour.

3.3 Traffic Execution and Evidence Pipeline

3.3.1 Temporal Scheduling and Stateful Execution

The generation of reproducible and stochastically sound workloads requires a strict operational separation between planned execution timelines and the actual runtime orchestration of transactions. This section details the execution engine of the Request Dataset Generator, analysing how it enforces temporal scheduling under concurrency limits and how the structured evidence pipeline processes, normalises, and stores resulting system events. By establishing these mechanisms, the discussion validates the client-side instrumentation before assessing live remote performance profiles.

Temporal model and occurrence generation

The timeline generator separates flow content from arrival time. It supports a constant rate, a configurable step profile, and a synthetic time-banded clinical profile. For a variable intensity $\lambda(t)$, arrivals are produced through Lewis–Shedler thinning: candidates are sampled from an upper-bound homogeneous process and retained according to $\lambda(t)/\lambda_{\max}$. Optional bursts add a configurable number of closely spaced occurrences around selected baseline arrivals. Every event records whether it is a base or burst arrival and, when applicable, its burst identifier.

The generator samples from the already constructed source dataset according to its empirical flow-type distribution. It therefore changes the temporal population without modifying transaction definitions. A seed makes workload and timeline sampling repeatable for a fixed configuration, but it cannot make the latency of live remote services deterministic. The nominal duration and rate also do not fix the final number of arrivals, which remains a realisation of the stochastic process. The generated schedule is therefore interpreted only as one reproducible sample from a declared stochastic distribution.

Scheduling and concurrency semantics

The execution engine follows scheduled timestamps using an asynchronous task per flow occurrence. A semaphore limits concurrent flows and encloses the whole flow rather than a single request. Within that boundary, steps are executed in order and share only their occurrence-local output context; separate flows can overlap. The engine stops a chain at the first non-success status or processing error of a flow, records the failing step, and leaves the remaining downstream steps unexecuted.

Live execution opens an HTTP client session, resolves dependencies, optionally uses the configured mTLS context, sends the request, extracts declared outputs, and validates required fields. Mock execution reads the synthetic response

attached to each step. A deterministic fault injector can apply declared error, timeout, or latency-spike rules by transaction, flow type, or step. These injected outcomes evaluate framework behaviour under controlled faults; they are not observations of provider reliability.

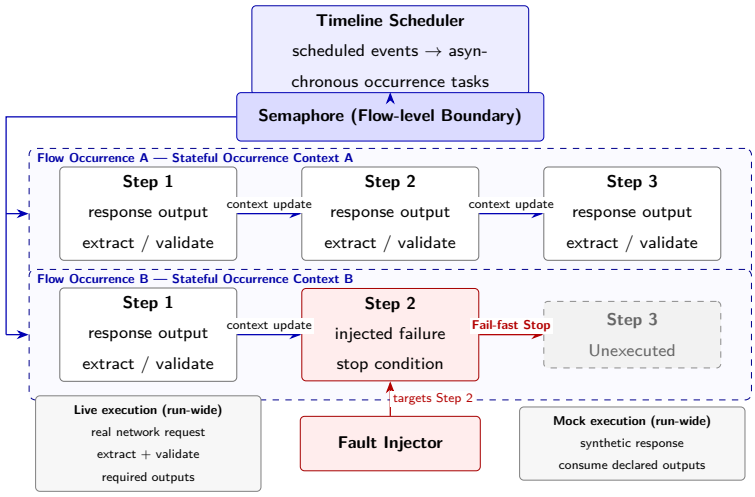


Figure 3.3: Asynchronous flow-execution model. Source: Author's elaboration based on the framework implementation.

3.3.2 Configuration Interface and Evidence Processing

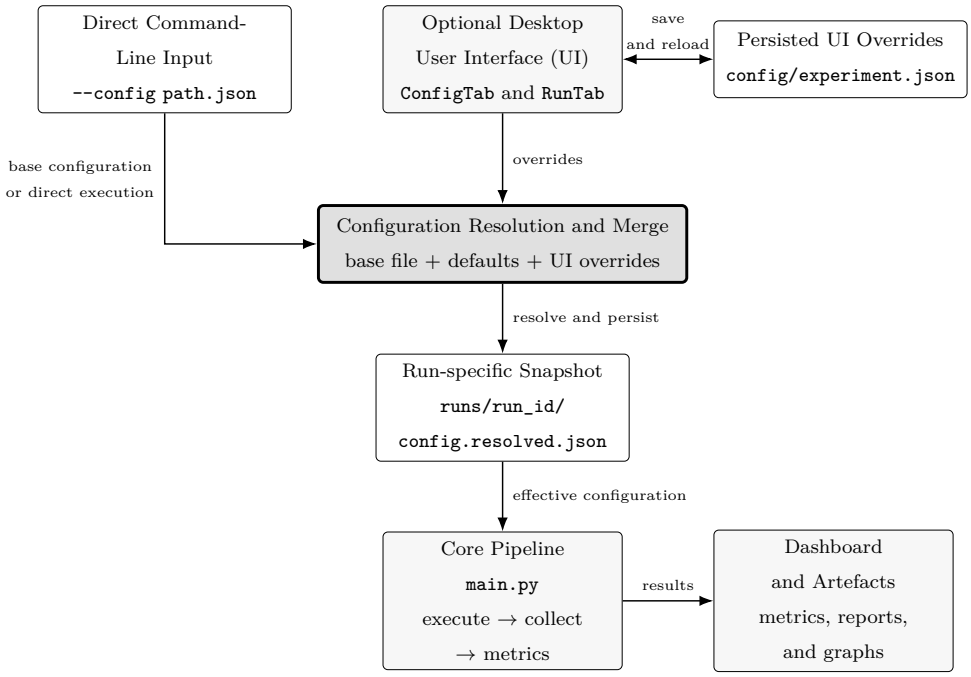


Figure 3.4: Experiment persistence and user-interface bridge architecture. Source: Author's design based on the framework's source code.

User-interface configuration and shared execution

The integration model treats the desktop user interface (UI) as an optional configuration front end to the same core pipeline used by direct execution. `ConfigTab` collects the experiment name, duration, arrival intensity, synthetic-patient count, active transactions, mediated-flow type, and target environment. The configuration bridge persists these choices as reusable overrides, allowing `RunTab` to reload them when the operator starts a new execution. The resolver combines the selected base JSON configuration, the framework defaults, and any explicit UI overrides into one effective configuration. It then stores the run-specific state in `config.resolved.json` before the pipeline consumes it, so repeatability depends on the resolved snapshot rather than on mutable state. The graphical path remains optional. The operator can bypass the UI and invoke `main.py` through the command-line interface (CLI), supplying the desired JSON file with `--config`. In this mode the selected file and the framework defaults feed the same configuration-resolution path without UI overrides. Both access modes therefore terminate in the same execution stages and artefact layout, while `DashboardTab` provides the graphical view of the resulting metrics, reports, and graphs when the desktop interface is used.

Structured execution events

The execution logger emits three JSONL event types. A `request_sent` event records experiment identifiers, flow occurrence, request and step identifiers, transaction, payload size, scheduled and actual start times, queue delay, active flows and steps, and burst provenance. A `response_received` event records status, response size, client latency, success, error, and fault-injection provenance. A `flow_completed` event records total duration, success, first failed step, declared step count, and burst provenance.

This schema implements the distinction between an interaction and an application flow established in Chapter 2. It also records enough provenance to

stratify observations by scenario, flow, transaction, arrival type, or execution.

Normalisation and surrogate enrichment

The collector parses the framework JSONL stream, pairs sent and received events by occurrence, reconstructs completed flows, and aggregates a bucketed time series. Its output contains four top-level parts: metadata, requests, flows, and timeseries. Request records retain client latency, outcome, concurrency, scheduling, size, and provenance. Flow records retain completion time, executed steps, skipped downstream steps, and outcome.

A second parser can ingest a compatible middleware JSONL stream. Matching records may add gateway and upstream latencies, retry count, circuit-breaker state, and infrastructure observations. When no middleware record is available, these fields remain null.

Metric and reporting engine

`MetricsCalculator` transforms the normalised request, flow, and time-bucket populations into eighteen measures grouped into performance, reliability, resilience, scalability, and observability. Performance separates Flow Completion Time (FCT) from step latency and reports request and flow throughput, burst and non-burst observations, and Path-level Overhead (ΔL) for compatible direct and mediated populations. Reliability retains error, timeout, retry, and flow-success rates; resilience derives recovery episodes, propagation, and bucket-level availability; scalability relates available resource and concurrency observations to load; observability measures event correlation and scheduling fidelity.

`StatisticsHelper` sorts each eligible population and calculates its size, mean, median, standard deviation, range, and P50, P75, P90, P95, and P99 through linear interpolation between adjacent ranks. Median, P95, and P99 retain the centre and tail of the empirical stochastic distribution, in line with the latency-analysis rationale established by Iorio et al. and Akram et al. The

`WarningDetector` then evaluates the calculated structure against configurable thresholds for tail latency, error rate, availability, resource saturation, and scheduling drift; it also flags a P99/P50 ratio above the configured skew rule. JSON and Markdown reports, CSV tables, and optional graphs preserve both the statistic and its population. These warnings automate inspection but remain software signals, not clinical, contractual, or production acceptance criteria.

Figure 3.5: End-to-end pipeline: from resolved configuration to quantile-based metric analysis. Source: Author's elaboration based on the framework's source code.

3.4 Evaluation Protocol and Experimental Design

3.4.1 Evaluation Questions and Validity Construction

Credible performance profiling within a constrained observation perimeter requires a rigorous evaluation protocol when the system remains a black box. This section delineates the concrete variables, controlled factors, and statistical treatments used to construct both internal and external validity. By matching scenarios across identical random seeds and isolating outcome populations, the experimental design ensures that the observed differentials between direct and mediated paths represent path-level displacements rather than being conflated with environmental noise.

Evaluation questions

The implemented architecture and the live evidence lead to four evaluation questions:

1. Can RDG preserve ordered transaction dependencies and isolate the state of concurrent flow occurrences during authorised live execution?

2. Can a declared flow mix and temporal model be reproduced with traceable identifiers, seeds, configuration, and artefacts?

3. Can execution events be transformed into consistent request, flow and time-window populations without filling unavailable middleware fields?

4. How do latency distributions, reliability, and flow completion vary with flow composition, offered load, bursts, and mediated execution?

The campaign answers these questions within the authorised client-side observation boundary. Mock mode supports internal code debugging only and does not contribute observations to the result population discussed in this chapter.

Validity construction

The pipeline separates domain contracts, temporal load, execution, and analysis while retaining every intermediate artefact. Flow Completion Time (FCT) measures the elapsed time between the first request and terminal event of one flow occurrence. Its distributions remain stratified by flow type and outcome because a rapid failure does not represent successful service performance. Path-level Overhead (ΔL) is admissible as a performance measure only when direct and mediated populations contain comparable, successful flows.
The execution semaphore applies to complete flows. This protects within-occurrence dependencies and makes the number of active flows explicit, but it also means that long or blocked flows retain a concurrency slot. This is part of the evaluated execution model and does not provide an unqualified model of provider-side concurrency. The client-side clock measures scheduling and request completion at the generator; internal service queues remain outside this boundary.

Advanced Black-box with Surrogate Observability

Framework evidence always records scheduled and actual dispatch, client-side step latency, response outcome, flow completion, and client-observed concur-

rency. Surrogate evidence may add middleware timing, retry state, circuit-breaker state, or infrastructure measures. A join remains valid only when identifiers, occurrence semantics, clocks, and measurement boundaries are compatible; join coverage is a data-quality measure rather than an assumption.

Four *surrogate observability invariants* govern every differential: occurrence identity remains stable across the artefact chain; compared timers use the same client-boundary definition; transport, application, and terminal flow outcomes remain distinct; and unavailable internal measurements remain null. Semantic equivalence, payload class, step count, connection policy, and load define additional matching conditions. A violation changes the estimand and prevents the resulting difference from being interpreted as Path-level Overhead (ΔL). Unsynchronised series are not aligned visually or subtracted across different clock and boundary semantics. A statistical association supports a diagnostic hypothesis, whereas causal attribution requires controlled variation and compatible measurements.

3.4.2 Variables and Reproducibility Controls

Analysing the performance of distributed workflows requires separating the active workload drivers from the static environmental parameters that constrain the system. To guarantee execution reproducibility across independent runs, the experimental controls are partitioned into three primary categories: independent workload factors, dependent system indicators, and invariant execution constants.

Variables and controlled factors

The independent variables exposed by the framework include execution mode, flow mix, source-dataset size, temporal profile and intensity, burst parameters, concurrency limit, timeout, and declared fault-injection rules. Dependent variables include step and flow latency distributions, request and flow throughput, error and timeout rates, flow-success rate, scheduling drift, queue delay, active

flows and steps, recovery episodes, and evidence completeness. Scenario configuration controls duration, concurrency, temporal profile, and flow mix. Paired scenario contrasts use the same seed, while payload and step-count differences remain explicit in the interpretation.

Reproducibility protocol and campaign controls

The final campaign comprises 74 accepted live runs across ten scenarios. Every accepted run retains the resolved configuration, generated dataset, timeline, execution log, normalised metrics, tabular exports, and report. Scenario, run, execution, flow-occurrence, flow, request, and step identifiers remain traceable across this chain. The campaign applies deterministic scenario ordering within each replicate and the seed rule `base_seed + replicate - 1`; equal replicate numbers therefore support paired scenario contrasts without merging independent flow occurrences.

Statistical treatment

The analysis reports request and flow populations, exact outcome rates, and successful FCT percentiles. Medians describe central behaviour, while P95 and P99 expose tail amplification. Pooled occurrence-level summaries quantify the complete evidence population; run-level summaries and equal-seed paired differences preserve the run as the experimental unit. Failed and successful flows remain separate, and a percentile is undefined when a scenario contains no successful flow. In this chapter, *quantile-based confidence* denotes the degree to which replicated, outcome-homogeneous populations support a consistent central or tail interpretation; it does not denote a parametric confidence interval. Confidence therefore decreases when repetitions are few, scenario populations are unbalanced, or the compared paths terminate with different outcomes.

3.5 Scenarios, Indicators, and Quality Controls

3.5.1 Scenario Evidence and Operational Populations

The transition from controlled nominal execution to system saturation requires a structured set of test profiles and multidimensional performance indicators. This section maps the concrete experimental scenarios used during the campaign, defining the operational metrics and quality-control thresholds that determine whether a run is accepted into the final quantitative evidence set. This systematic screening establishes a clean dataset and safeguards the subsequent statistical interpretations from data-quality anomalies.

Declared scenarios

The final inventory covers constant baselines, bursts, step-load ramps, extreme stress, fault injection, direct and mediated flow comparisons, and a compact live smoke profile. Every accepted resolved configuration declares live execution. The chapter therefore treats the inventory as *Live Production-Like Scenarios*. The full flow mix includes context operations, direct patient query and creation, ITI-18 and ITI-43 transactions, the mediated patient path, and the Scryba Sign `GetUserInfo`/`SignOneDoc` sequence.

Table 3.3: Experimental-campaign inventory. Source: Author's elaboration based on the final experimental campaign.

Scenario	Live runs	Temporal profile	Principal flow population
`baseline`	5	daily clinical	full seven-flow mix
`baseline_constant`	5	daily clinical	full seven-flow mix
`burst`	5	daily clinical with bursts	full seven-flow mix
`direct_vs_middleware`	14	daily clinical	direct patient and mediated paths
`error_injection`	6	daily clinical	full mix with declared faults
`flow_type_comparison_direct`	6	daily clinical	direct, ITI, and Scryba Sign flows
`flow_type_comparison_middleware`	15	daily clinical	mediated patient path
`live_smoke_small`	8	daily clinical	compact full-flow mix
`load_ramp`	5	step	full seven-flow mix
`stress_extreme`	5	step with bursts	full seven-flow mix

Table 3.3 defines the final accepted evidence set. The distribution of runs reflects scenario-specific validation depth: the comparison families receive more repetitions, while the high-volume load profiles obtain large within-run populations. Scenario inference therefore uses its own observed population rather than weighting all scenarios as if they contain equal run counts.

Operational performance populations

Flow completion time is calculated over completed flow records and remains stratified by flow type and outcome. Step latency is calculated over paired request/response occurrences and remains stratified by transaction and flow type. Request throughput counts completed requests per observed second; flow throughput counts completed flow occurrences over the same run interval. The two are reported together because flow types have different step counts. Reliability uses request errors, timeouts, and completed-flow outcomes. Failure at a step is associated with skipped downstream steps rather than fabricated requests. Scheduling fidelity uses the difference between planned and actual client dispatch together with queue delay and active-flow/step counts; finally, burst analysis uses the explicit base/burst labels. Middleware breakdown, resource saturation, and retry or circuit-breaker measures are valid only for records actually enriched by a second source.

3.5.2 Quality Controls and Acceptance Boundary

Before the captured event traces enter the statistical analysis, each completed run must pass a series of deterministic validation checks. These quality controls ensure that the network state, transaction integrity, and metadata records conform to the experimental configuration. The evaluation relies on a multistage validation pipeline that assesses the following criteria:

Data-quality controls

Before interpretation, each run is checked for:

- an available resolved configuration and explicit execution mode;
- consistent scenario, run, execution, flow-occurrence, flow, request, and step identifiers;
- paired sent/received events and an explicit flow-completion event;
- recorded population sizes, time range, units, and flow-type coverage;
- explicit fault-injection and burst provenance;
- exclusive inclusion of live populations in final results;
- retained failed observations rather than silent imputation.

Across the 74 runs, RDG records 757,517 structured events, 299,534 paired request occurrences, and 158,449 flow completions. Every run satisfies the event identity $N_{events} = 2N_{requests}+N_{flows}$; all required identifiers agree with the resolved configurations. Middleware-enriched fields remain empty, consistently with the authorised client-side boundary. The dataset therefore supports end-to-end client observation but not internal cross-layer decomposition.

Acceptance criteria

Research acceptance requires a coherent artefact chain, explicit outcomes, preserved failure observations, reproducible configuration, and a stated observation boundary. The 74-run dataset satisfies these criteria and constitutes the final quantitative evidence population for the thesis.

3.6 Empirical Stochastic Distributions

3.6.1 Outcome-Stratified Evidence

This section presents the primary empirical findings collected during the 74 accepted live runs of the campaign. The resulting distributions characterise the performance profile of the healthcare workflows under varying load conditions, exposing how tail latency, error rates, and scheduling drift behave as

joint stochastic properties of the system. To guide the reader through this multidimensional analysis, the discussion is structured around three core perspectives:

- **Outcome-Stratified Populations.** Section 3.6.1 establishes the baseline validation of the collected dataset and identifies the global ratios between completed and failed flows across the campaign.
- **Load Dynamics and Saturation Boundaries.** Sections 3.6.2– 3.6.3 analyse the onset of queue backpressure and the resulting delivery drift under extreme stress, thereby defining the physical limits of the cooperative event loop.
- **Flow Heterogeneity and Path Displacement.** Sections 3.6.4– 3.6.5 isolate the distinct latency signatures of individual transaction families and quantify the systematic apparent path-level overhead associated with regional mediation.

Empirical population and evidence-chain validity

The 74 accepted live runs instantiate the complete evidence chain from generation and timeline construction to asynchronous execution, structured collection, normalisation, and metric calculation. The resulting sample space contains 299,534 request occurrences, of which 219,906 succeed (73.416%), and 158,449 flow occurrences, of which 78,821 reach a successful terminal state (49.745%). These populations define outcome-conditioned random variables rather than a test-coverage inventory: Flow Completion Time (FCT) is therefore characterised separately for successful and interrupted occurrences, and the path comparison yields a Path-level Overhead (ΔL) of 184.7 ms at the authorised client boundary.

Table 3.4: Live scenario populations and successful-flow Flow Completion Time (FCT) quantiles at P50, P95, and P99. Source: Author's elaboration based on the final experimental campaign.

Scenario	Runs	Requests	Flows	Flow success %	FCT P50 ms	FCT P95 ms	FCT P99 ms
baseline	5	594	307	54.07	559.7	1,211.8	18,892.6
baseline_constant	5	2,771	1,431	53.88	504.5	1,093.0	18,271.8
burst	5	20,731	10,562	52.14	461.9	1,129.0	22,759.4
direct_vs_middleware	14	1,590	1,590	15.72	755.8	1,658.8	17,988.0
error_injection	6	1,722	868	58.87	543.5	2,840.0	21,547.8
flow_type_comparison _direct	6	2,251	1,037	80.42	571.1	17,966.6	21,934.5
flow_type_comparison _middleware	15	1,687	1,687	15.63	755.8	1,658.8	17,988.0
live_smoke_small	8	196	122	42.62	755.8	1,658.8	17,988.0
load_ramp	5	11,750	6,066	53.58	456.0	948.4	23,174.6
stress_extreme	5	256,242	134,779	50.25	572.3	2,026.9	21,505.9

3.6.2 Load-Scheduling Calibration

The linear load ramp scenario works as the calibration condition introduced in Section 2.1.2: the generator remains credible only if its timing error stays negligible before offered work reaches the declared concurrency boundary. Figure 3.6 therefore reads client-observed active work together with the ninety-fifth-percentile queue delay.

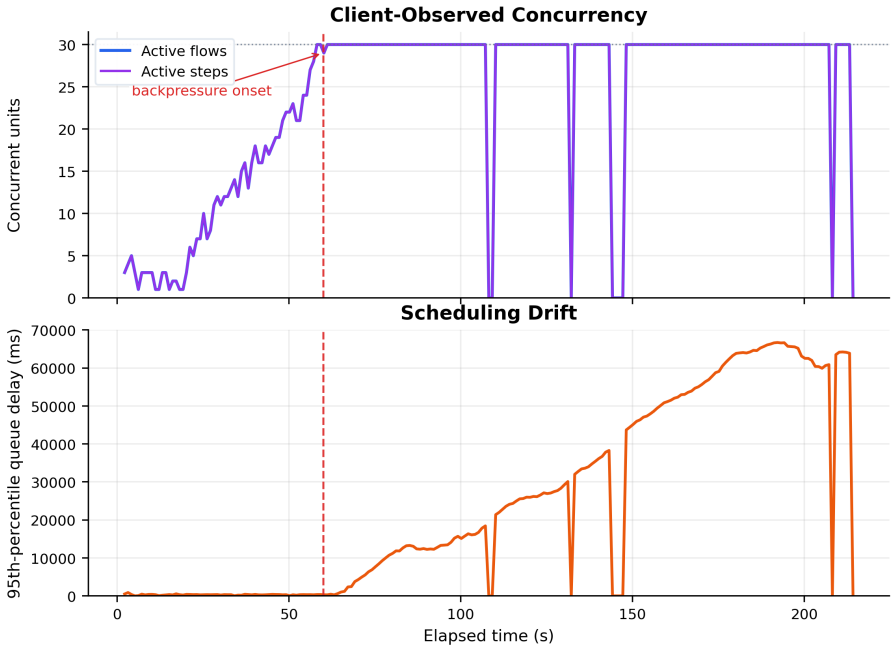


Figure 3.6: Client-observed concurrency and scheduling drift. Source: Author's elaboration based on the experimental campaign.

During the first minute, the queue-delay series remains close to zero; the rising envelope begins only when the active-flow and active-step series reach the configured limit of 30. The plateau at 30 occurs at approximately 60 seconds, and the persistent queue-delay increase starts within the following buckets. Its envelope then grows to about 67 seconds before the arrival stream drains. This temporal ordering is consistent with semaphore backpressure: the execution semaphore wraps an entire flow occurrence, so a flow that waits for admission accumulates dispatch delay while admitted flows preserve their internal transaction order. The graph consequently shows controlled queue formation at the client boundary, not evidence that the event loop loses timing fidelity before saturation.

The isolated returns to zero in the queue-delay series coincide with buckets that contain no eligible delay observation; they do not reverse the rising envelope. Likewise, the plateau records the configured client-side admission limit and does not identify a provider-side capacity threshold. The paired panels thus validate the generator's calibration logic within the declared measurement boundary: timing remains stable below the limit, while overload is made explicit as measurable queueing rather than hidden inside request latency.

Each load profile is paired with the `baseline_constant` run that uses the same seed. At run level, the median P50 difference equals −43.9 ms for `burst` (−8.84%; interquartile range (IQR): −69.1 to −41.6 ms) and −58.7 ms for `load_ramp` (−10.32%; IQR: −69.0 to −30.0 ms). These profiles shift the centre without producing a uniform rightward displacement of their empirical stochastic distributions. In contrast, `stress_extreme` increases the median run-level P50 by 55.3 ms (+10.82%), with an IQR from −0.02 to +125.9 ms. Over the pooled successful-flow population, P50 rises from 504.5 to 572.3 ms (+13.45%), P95 rises from 1,093.0 to 2,026.9 ms (+85.44%), and P99 rises from 18,271.8 to 21,505.9 ms (+17.70%), while flow success decreases by 3.63 percentage points. The non-uniform quantile displacement shows that extreme load changes the distributional shape: its principal effect appears around the upper-tail shoulder

captured by P95, while the far tail is already populated by structurally long flows in the baseline.

3.6.3 Stochastic Saturation and Operational Response

Figure 3.7 aligns concurrency, scheduling drift, completion throughput, error rate, and error-free bucket availability from the extreme-stress run on a common temporal axis. The request-latency distribution remains separate in Figure 3.8, because its horizontal axis represents latency rather than elapsed experimental time. The histogram is bimodal, but its secondary concentration lies near 45 seconds rather than 30 seconds. This position agrees with the run-specific 45-second step timeout and coexists with a declared five-percent injected-timeout rule. The visual evidence therefore supports a timeout-bound mode.

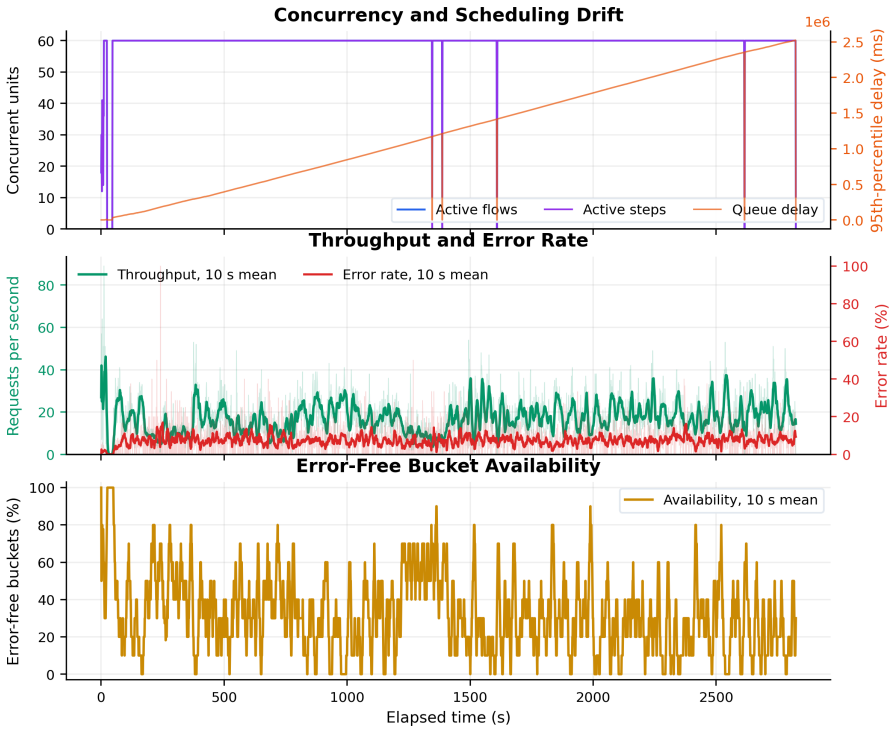


Figure 3.7: Three-panel operational timeline under stochastic queueing saturation. Source: Author's elaboration based on the final experimental campaign.

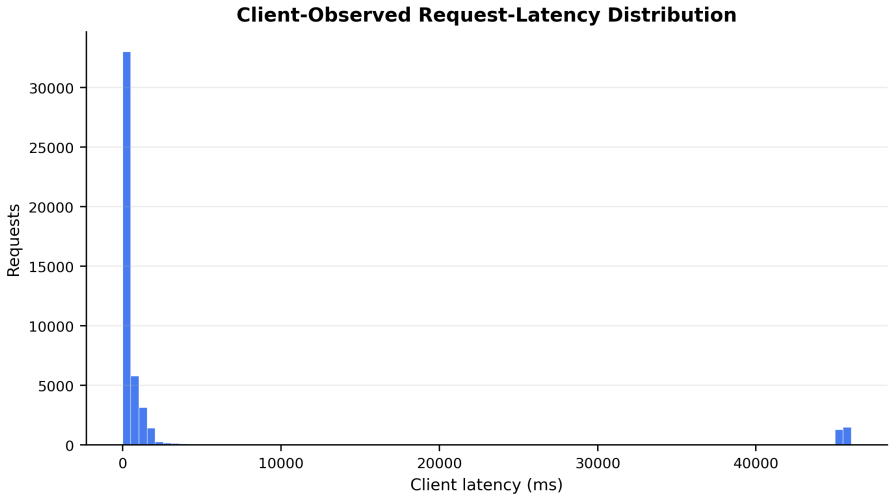


Figure 3.8: Isolated client-observed request-latency distribution under stochastic queueing saturation. Source: Author's elaboration based on the final experimental campaign.

The independent pooled analysis places the successful Flow Completion Time for the IHE-XDS.b transaction (ITI-18) at 37,869.5 ms for the ninety-fifth percentile and 43,972.1 ms for the ninety-ninth percentile. Those values characterise an outcome-stratified flow population, whereas the histogram pools request latencies from one run. They nevertheless locate the reported tail in the same tens-of-seconds regime as the timeout-bound mode, subject to the different populations and timers.

The operational panels derive from the same one-second bucket boundaries. Ten-second rolling means expose the macroscopic trajectory, while transparent raw throughput and error-rate observations preserve the underlying variation. Error bursts coexist with positive completion throughput across most of the run. This joint view distinguishes useful completions from error incidence, but it does not by itself estimate a correlation coefficient or establish causation. In particular, the smoothed availability series averages a binary indicator that equals 100 only when a bucket has no errors and zero otherwise; it remains an experimental bucket indicator rather than a continuous estimate of provider availability.

In Akram et al.'s stochastic model, the conjunction of an expanding latency

tail and diminishing useful completions constitutes empirical evidence of the closing of the Load-Sharing Window and hence of the onset of the risk of *Transfer in Vain*. The `stress_extreme` population exhibits this joint signature at the client boundary through the 85.44% P95 amplification, the positive P99 displacement, and the lower flow-success rate. This language transfers the explanatory model rather than an internal topology: the Load-Sharing Window remains a methodological abstraction, because the evidence does not observe a regional load-transfer mechanism directly. In Iorio et al.'s terms, the same signature appears in application-level FCT rather than in network round-trip time, which gives quantile-based confidence that the effect belongs to complete-workflow behaviour within the observed population.

3.6.4 Domain-Specific Flow Signatures

A domain-specific workload preserves transaction chains that a generic request generator otherwise collapses into an undifferentiated latency sample. In Figure 3.9, different integration flow types occupy visibly different locations and spreads. The figure thus supports a family of conditional random variables $L \mid$ flow type rather than a single unconditional latency variable.

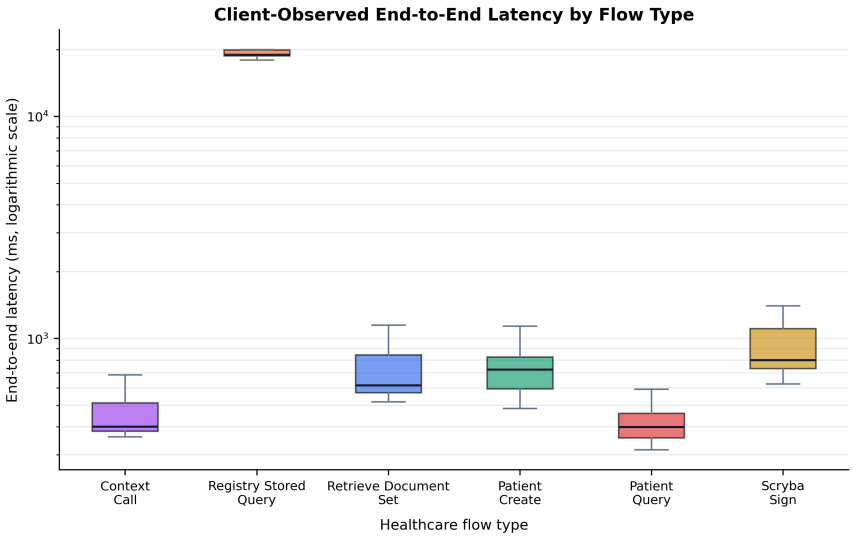


Figure 3.9: Client-observed end-to-end completion time by flow type. Source: Author's elaboration based on the final experimental campaign.

The XDS.b Registry Stored Query flow is centred near 19 seconds in the selected run, whereas the displayed Anagrafe Zero patient-query and patient-creation paths remain predominantly below about 1.5 seconds. The repository-query, context-call, and signature distributions form further distinct bands. These are visual, single-run comparisons without outcome stratification; the boxplot therefore establishes heterogeneity but does not replace the pooled percentiles reported in Table 3.5.

This separation has methodological consequences. Fast Healthcare Interoperability Resources (FHIR) Representational State Transfer (REST) patient-registry interactions and Integrating the Healthcare Enterprise (IHE) Cross-Enterprise Document Sharing (XDS) Simple Object Access Protocol (SOAP) transactions differ in protocol, payload, dependency chain, and terminal condition. Averaging them erases precisely the semantic structure needed to interpret application-level completion time. Stratification by flow type is therefore part of the measurement model, not a presentation choice.

Outcome-stratified transaction and flow distributions

Table 3.5 separates transaction outcomes from successful end-to-end flow populations. The separation defines distinct stochastic variables for technical attempts and useful terminal work, because a prerequisite can terminate an occurrence before the focal transaction starts. ITI-18 contributes 21,722 transaction outcomes and a 15.63% success rate; its successful FCT distribution also displays the longest P95 and P99 tails. ITI-43 contributes 21,514 outcomes with 99.34% transaction success. Scryba Sign records 17,838 successful `GetUserInfo` calls and 17,015 successful `SignOneDoc` calls, which yields 17,015 successful complete signature flows. The values provide large empirical populations for FCT and tail analysis while preserving the exact outcome semantics of each step.

Table 3.5: Transaction-outcome populations and successful end-to-end Flow Completion Time distributions at P50, P95, and P99. Source: Author's elaboration based on the final experimental campaign.

Flow family	Transaction outcomes / successes	Successful flows	FCT P50 ms	FCT P95 ms	FCT P99 ms
ITI-18	21,722 / 3,395 (15.63%)	3,395	1,518.6	37,869.5	43,972.1
ITI-43	21,514 / 21,371 (99.34%)	21,371	579.7	1,559.2	3,992.1
Scryba Sign	`GetUserInfo`: 17,838 / 17,838; `SignOneDoc`: 17,838 / 17,015 (95.39%)	17,015	819.6	2,472.0	13,006.9

3.6.5 Client-Boundary Path Displacement

Figure 3.10 separates the path distributions by terminal outcome. The successful- and failed-path boxes pool the corresponding observed flows from all 14 complete direct-versus-mediated runs and retain their measured distributional shapes.

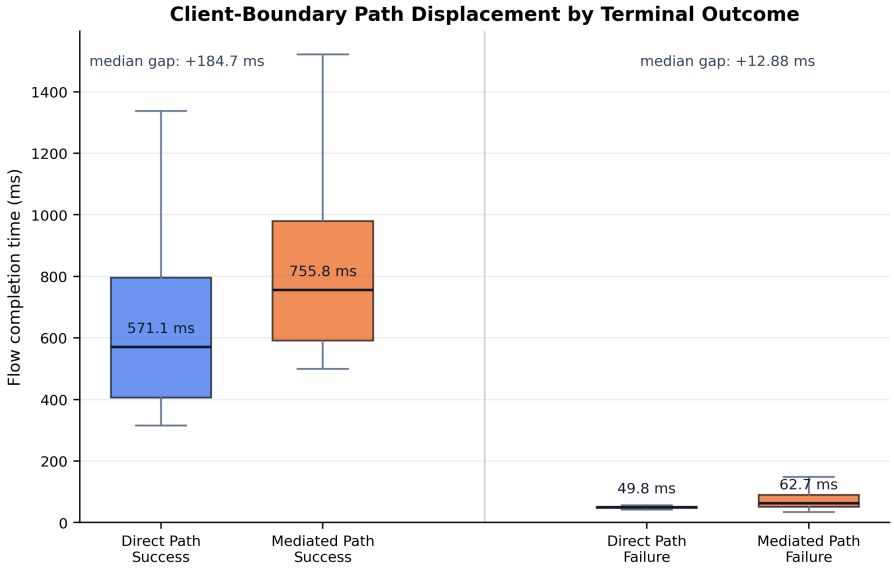


Figure 3.10: Client-observed path-level displacement stratified by successful completion and fail-fast termination. Source: Author's elaboration based on the final experimental campaign.

The independent pooled analysis compares the 755.8 ms mediated median with the 571.1 ms direct median and defines a client-observed *Path-level Overhead*

(ΔL) of 184.7 ms. This value remains an aggregate external displacement under the Surrogate Observability Strategy; it is not a decomposed middleware processing time. For the failed-flow population, the corresponding mediated-minus-direct differences equal +62.18 ms for the mean, +12.88 ms for the median, and +284.30 ms for the ninety-fifth percentile.

The 12.88 ms failed-flow median differential is the empirical signature of the execution engine's fail-fast rule: steps execute sequentially and the chain terminates at the first non-successful transaction. Let $X_0 \to X_1 \to \cdots \to X_m$ denote the ordered workflow states and let τ be the first failed transition. The engine acts as a stochastic truncation operator

$$\mathcal{T}_\tau(X_0, \ldots, X_m) = (X_0, \ldots, X_\tau), \qquad W_{\text{avoided}}(\tau) = \sum_{j=\tau+1}^{m} D_j, \tag{3.1}$$

where D_j denotes the service demand that downstream step j would inject. The observed low differential is consistent with a short realised prefix, whereas the implementation-level stopping rule establishes that the remaining suffix is not submitted.

This operator provides the execution-layer counterpart to the stochastic state-transition model in Figure 2.3. The two state spaces remain distinct: the figure describes transitions among node load states, while $\mathcal{T}_\tau$ acts on the dependency states of one healthcare workflow. Their connection is the avoided service demand. When a failed prerequisite suppresses downstream IHE-XDS.b Registry Stored Query [ITI-18] or Retrieve Document Set [ITI-43] transactions in flow families that contain them, the client observes only the truncated completion time and the channel receives no requests whose semantic prerequisites are already invalid. Under saturation, this avoided wasted work reduces the offered load that can drive the gateway from a normal toward an overloaded state and therefore acts as a protective mechanism against overload collapse. It does not, however, identify an internal gateway component or prove a provider-side capacity threshold. The centre–tail distinction follows the application-level Flow

Completion Time hierarchy of Iorio et al. and the stochastic quantile interpretation of Akram et al.; the surrogate observability invariants prevent either path displacement from being recast as an unsupported causal decomposition[2].

3.6.6 Synthesis of Answers to Research Questions

The empirical distributions and path-level differentials quantified in the preceding sections provide the evidence required to resolve systematically the core research questions posed in Chapter 1. Rather than evaluating latency as an isolated, static scalar, this synthesis reconciles the measurement observations with the stochastically bounded theoretical invariants of the system, verifying how workflow dependencies and saturation boundaries manifest in the experimental campaign.

Validation of the stochastic propositions

The quantitative evidence validates the methodological propositions of Chapter 2 within the observed perimeter. First, latency is an empirical stochastic distribution rather than a scalar property: P50, P95, and P99 move by different proportions under load, and the mean alone cannot represent that deformation. Second, application-level FCT remains the relevant unit for dependent workflows: the distinct ITI-18, ITI-43, and Scryba Sign tails show that request populations cannot substitute for complete-flow behaviour. Third, useful completion and latency require joint interpretation: the `stress_extreme` P95 amplification and success-rate reduction instantiate the closing-window signature formalised through Akram et al.'s Load-Sharing Window and Transfer in Vain model. Finally, occurrence identifiers and outcome stratification prevent repeated source flows, interrupted chains, and successful terminal work from collapsing into one heterogeneous sample.

[2]Iorio, Risso, and Casetti, "When Latency Matters: Measurements and Lessons Learned"; Akram, Akram, and Rehman, "Numerical Evaluation of Network Latency and Throughput in Distributed Systems".

Methodological consequences

The validation depends on invariants that also delimit every admissible generalisation. Transaction, step, flow, and occurrence identities remain continuous from generation through reporting; step order and dependency propagation remain sequential within an occurrence, whereas concurrency applies between independent flows. Resolved configuration, seed, execution mode, and population definition accompany each estimate. Request and flow throughput remain joint but non-interchangeable measures because the flow families contain different numbers of steps, while debugging populations remain outside the live empirical distributions.

The same discipline constrains the Path-level Overhead: semantic equivalence, step count, payload class, connection policy, load, and observation boundary define the estimand before subtraction. Configurable software warnings do not become acceptance thresholds, and flow mixes or arrival profiles do not become representative merely because they generate large samples. These conditions operationalise the surrogate observability invariants and preserve the distinction between a reproducible client-boundary characterisation and an unsupported causal decomposition.

3.7 Summary

The framework translates documented healthcare workflows into configurable, stateful, and asynchronous workloads, connecting transaction semantics with an executable performance experiment. Its evidence chain retains resolved configuration, deterministic seeds, flow-occurrence identifiers, timeline, execution events, normalised records, and final metrics. Every result therefore remains traceable to its scenario and run. This continuity answers the reproducibility and observability components of the research question.

The final campaign comprises 74 runs, 299,534 request occurrences, and 158,449 flow completions across ten live scenarios. Within this sample, the 21,722 ITI-

18 outcomes, 21,514 ITI-43 outcomes, 17,838 successful `GetUserInfo` calls, and 17,015 successful `SignOneDoc` calls form distinct transaction, flow, and outcome distributions. Their analytical value lies in the joint observation of Flow Completion Time (FCT), success, request and useful-flow activity, and P50, P95, and P99 rather than in volume alone.

Across load profiles, burst and load ramp do not produce a uniform increase in central FCT, whereas extreme stress raises pooled successful FCT by 13.45% at P50, 85.44% at P95, and 17.70% at P99, with a 3.63-percentage-point reduction in flow success. The asymmetric movement locates the dominant effect in the upper-tail shoulder rather than in a uniform translation of the distribution. This joint latency–success rate signature provides quantile-based confidence for a closing Load-Sharing Window and the associated risk of Transfer in Vain within the observed population, while the limited number of complete repetitions prevents extrapolation to a universal saturation or capacity threshold.

The differential component completes the synthesis. The direct–mediated contrast identifies a client-observed Path-level Overhead (ΔL) of 184.7 ms at P50 within the authorised boundary. The surrogate observability invariants make outcomes, centre, tail, load sensitivity, and path displacement jointly interpretable. The resulting evidence answers Chapter 1's research question within the stated perimeter and connects Chapter 2's stochastic theory to the empirical distributions produced by the framework.

Chapter 4

Conclusions

This chapter closes the traceability chain that begins with the healthcare interoperability problem in Chapter 1, acquires its analytical foundations in Chapter 2, and produces empirical evidence in Chapter 3. Its objective is to synthesise the research contributions, reconcile the observations with the stochastic invariants that delimit their interpretation, assess the limitations of the available evidence, and derive a concrete engineering roadmap from the measured behaviour. The resulting synthesis treats performance as a joint property of latency distribution, useful completion, admission control, and workflow state: no single scalar replaces this joint view, and no client-side measurement is presented as an unsupported decomposition of a protected regional platform.

4.1 Connecting Objectives to Evidence

4.1.1 Overview

Chapter 1 defines four objectives: interpret the selected standards and architectures, formalise the corresponding healthcare workflows, translate them into executable and observable workloads, and compare direct with gateway-mediated paths. The Request Dataset Generator (RDG) provides the artefact that connects these objectives. Its evidence chain preserves configuration, seed, timeline, occurrence identity, request and flow outcomes, and statistical population from workload construction to reporting. The three results below provide the principal closure: path-level displacement characterises the externally visible difference between the declared path populations, stochastic truncation explains the different completion behaviour of interrupted flows, and queueing saturation connects client-side delivery drift with the deformation of useful completion under extreme offered load. Table 4.1 provides the corresponding objective–model–evidence map.

Table 4.1: Structural reconciliation matrix mapping the thesis objectives to the formal invariants of Chapter 2 and the empirical findings of Chapter 3. Source: Author's elaboration.

Research Objective	**Formal Invariant or Model**	**Empirical Evidence**	**System Status or Outcome**
Interpret the selected standards and architectures.	Occurrence-local dependency model and surrogate observability boundary.	Executable regional flow families preserve security, identity, document, and signature prerequisites across 74 complete live runs.	Reconciled within the Veneto integration perimeter; external validity remains scoped.
Formalise healthcare workflows.	Sequential intra-flow states and stochastic truncation $\mathcal{T}_\tau$ at the first unsuccessful transition.	The failed mediated-minus-direct population has a median differential of +12.88 ms; the invalid suffix receives no dispatch.	Fail-fast behaviour is supported; failed-flow tail dispersion remains visible.
Translate the workflows into executable and observable workloads.	Evidence-chain identity, planned-versus-actual dispatch, and a configured flow-concurrency bound.	The final dataset contains 757,517 structured events and 158,449 flow completions; active flows plateau at 30 while drift grows under saturation.	Instrument validity holds below the admission boundary; overload exposes client-side backpressure.
Compare direct and gateway-mediated paths.	Path-level Overhead $\Delta L(P_{50}) = L_{\text{mediated}} - L_{\text{direct}}$.	Successful-flow medians of 755.8 ms and 571.1 ms $\Delta L(P_{50}) = 184.7$ ms.	The client-boundary displacement is quantified; mediation-only and component-level attribution remain non-assessable.

4.1.2 Mediation Overhead and Path-Level Displacement

The fourth objective of Chapter 1 requires a differential measurement because internal gateway telemetry lies outside the authorised observation perimeter. Chapter 2 consequently defines the Path-level Overhead for a declared statistic q as

$$\Delta L(q) = L_{\text{mediated}}(q) - L_{\text{direct}}(q), \tag{4.1}$$

subject to compatibility of transaction semantics, payload class, terminal outcome, connection policy, offered load, and observation window. This definition fixes the estimand before interpreting its magnitude.

The distinction between network, transport, and application-level completion time developed by Iorio et al. determines the observation boundary. The client observes the complete path effect, whereas the available evidence does not isolate one internal processing stage[1].

Chapter 3 instantiates this estimand at the median. For successful flows, the declared mediated population has a median completion time of 755.8 ms and the declared direct comparison population has a median of 571.1 ms. Their empirical displacement is

$$\Delta L(P_{50}) = 755.8\,\text{ms} - 571.1\,\text{ms} = 184.7\,\text{ms}. \tag{4.2}$$

The principal quantitative finding is therefore *a successful-flow median displacement of* $\Delta L(P_{50}) = 184.7$ *ms*. This value places the centre of the observed mediated distribution to the right of the direct comparison distribution. It does not imply that every request incurs the same displacement.

The interpretation remains constrained by the implemented flow definitions. Chapter 3 establishes that the direct patient paths contain two steps, whereas the mediated patient workflow contains five steps and different contracts. The subtraction consequently describes the declared campaign populations; it does

[1] Iorio, Risso, and Casetti, "When Latency Matters: Measurements and Lessons Learned".

not constitute a single-variable estimate of a semantically equivalent operation or an operational budget attributable only to gateway mediation.

Network transit, connection and security handling, policy enforcement, transformation, routing, queueing, downstream execution, retries, and the different workflow compositions can all contribute to the aggregate difference. The result remains informative as a client-boundary path displacement while respecting the surrogate-observability invariant established in Chapter 2.

The positive median displacement also does not demonstrate execution instability or stability on its own. Those interpretations require joint inspection of centre, tail, success rate, and delivery drift within a specified flow mix and load regime. The thesis therefore retains the measured distributional difference while separating it from mediation-only and component-level causal attribution.

4.1.3 Asynchronous State Truncation and Fail-Fast Efficiency

The framework preserves a strict execution invariant: steps within one flow occurrence execute sequentially, while independent occurrences execute concurrently. Let an occurrence traverse the ordered states $X_0 \to X_1 \to \cdots \to X_m$, and let the random index τ identify the first unsuccessful transition. The execution engine implements the stochastic truncation operator

$$\mathcal{T}_\tau(X_0, \ldots, X_m) = (X_0, \ldots, X_\tau), \tag{4.3}$$

so the realised workflow contains only the valid prefix.

If D_j denotes the service demand associated with downstream state j, the fail-fast rule suppresses

$$W_{\text{avoided}}(\tau) = \sum_{j=\tau+1}^{m} D_j. \tag{4.4}$$

The operator is stochastic because the position and cause of the first failure vary across occurrences. Once τ occurs, the stopping rule remains deterministic

and the suffix receives no dispatch.

The outcome-stratified path comparison supplies the empirical counterpart to this model. Successful mediated flows exhibit a median path displacement of +184.7 ms, whereas failed mediated flows exhibit *a median differential of* +12.88 *ms* relative to failed direct flows. The mean and ninety-fifth-percentile failed-flow differentials, respectively +62.18 ms and +284.30 ms, preserve the non-negligible tail of the failure population.

Figure 4.1 condenses this outcome-conditioned contrast into the representative direct patient-query state progression.

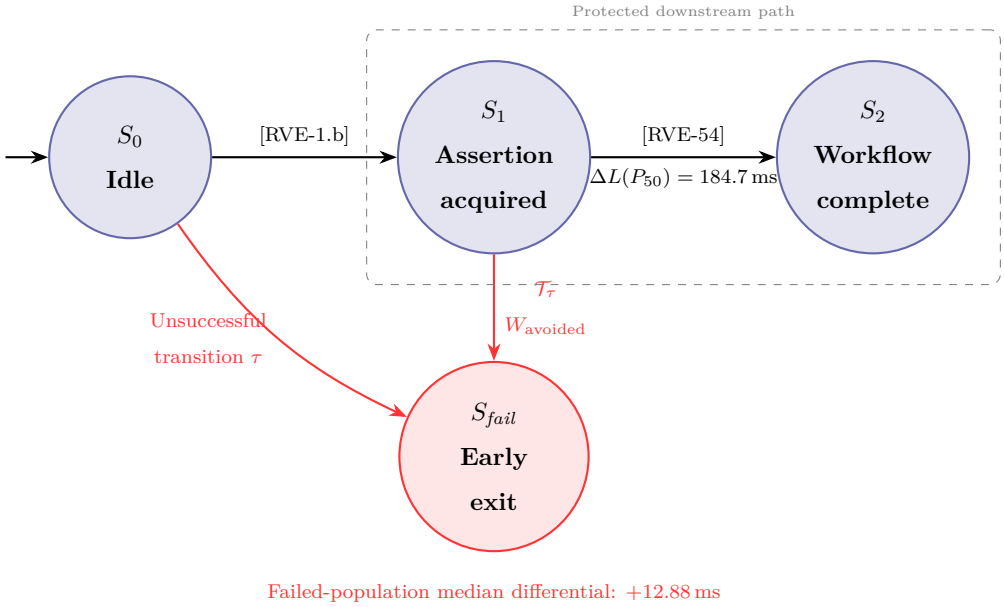


Figure 4.1: Empirically annotated workflow automaton representing transaction-state progression, stochastic truncation $\mathcal{T}_\tau$, and early execution termination. Source: Author's elaboration.

The contrast between +12.88 ms and +184.7 ms is consistent with a shorter realised prefix for at least half of the interrupted occurrences. It does not state that failure occurs within 12.88 ms: the measured quantity is the median mediated-minus-direct differential within the failed-flow population.

The normal branch in Figure 4.1 advances from S_0 to S_1 when [RVE-1.b] produces the Security Assertion Markup Language assertion required by the protected call. A successful [RVE-54] transition then reaches S_2. Its 184.7 ms

annotation reports the median displacement of the successful path populations; it is neither a transition-time bound nor a probability attached to the automaton.

An unsuccessful prerequisite instead reaches S_{fail} and suppresses the remaining dependency suffix. For flow families that contain document transactions, this branch prevents semantically invalid Integrating the Healthcare Enterprise IT Infrastructure transaction 18 or transaction 43 requests from entering the downstream path. When the security prerequisite already fails, those unexecuted requests also avoid the run-specific 45-second step-timeout regime. The figure assigns no transition probability because the campaign reports outcome-conditioned populations rather than a fitted probability for one common automaton.

Stochastic truncation connects the workflow state model to the load-state model through avoided demand without conflating their state spaces. The former describes dependency progression inside one healthcare occurrence; the latter, adapted from Akram et al., relates traffic intensity, latency, and the validity of distributed state under load[2].

At the gateway boundary, suppression of an invalid suffix removes requests that would otherwise compete for admission, connection handling, parsing, validation, transformation, and routing. At the downstream boundary, services do not receive requests whose inputs depend on a failed state. This reduction in submitted work can weaken one path to queueing backpressure, but the campaign does not isolate its marginal effect on congestion. The mechanism therefore establishes fail-fast termination as an execution and correctness property; it neither reveals gateway capacity nor eliminates transfer in vain for requests that already enter a long queue or approach a timeout.

[2]Akram, Akram, and Rehman, "Numerical Evaluation of Network Latency and Throughput in Distributed Systems".

4.1.4 Queueing Saturation and Load-Sharing Window

Chapter 1 asks whether the experimental method exposes behaviour at extreme offered load rather than only nominal latency. The calibration evidence in Chapter 3 locates the relevant transition at the client-side admission mechanism, when *active flows reach the configured capacity* $C_{\max} = 30$ *at approximately* 60 *s*. The scheduling-drift envelope then passes approximately 60,000 ms and approaches 67,000 ms before the arrival stream drains. Flow occurrences continue to arrive, but the executor cannot admit them at their planned instants; delivery drift therefore accumulates outside the semaphore.

The plateau identifies the generator's configured concurrency boundary, not a regional service capacity of 30. Once the semaphore remains occupied, the planned arrival process $\lambda(t)$ no longer equals the workload delivered to the remote path. Request latency excludes the interval spent waiting for local admission, whereas scheduling drift records the difference between planned and actual dispatch. Reading the two measures together therefore separates offered work from admitted work and prevents late dispatches from being reclassified as normal arrivals.

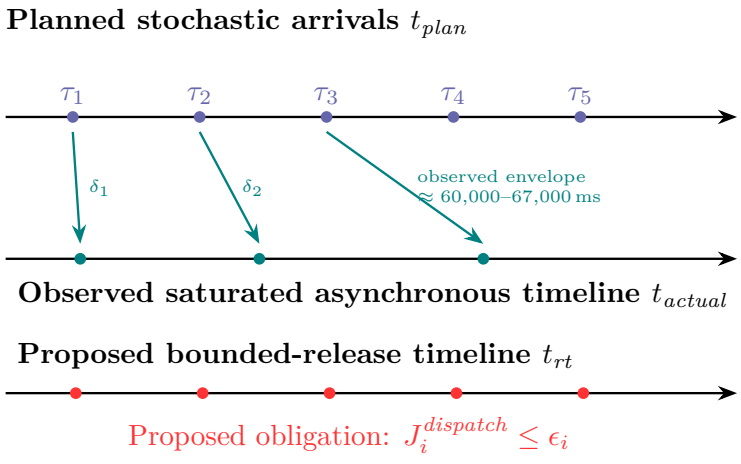


Figure 4.2: Comparative timing chronogram separating the planned arrival timeline, observed delivery drift, and the bounded-release real-time dispatcher. Source: Author's elaboration.

Figure 4.2 represents this distinction as a chronogram. The upper timeline contains the planned occurrences generated from $\lambda(t)$; the middle timeline shows

their progressively delayed admission under cooperative event-loop saturation. The lower timeline states the proposed future obligation for a regulated real-time dispatcher: each release remains within a declared bound ϵ_i.

The extreme-stress scenario adds the remote-path and completion evidence needed for stochastic interpretation. Relative to the paired constant baseline, pooled successful Flow Completion Time rises by 13.45% at the fiftieth percentile, 85.44% at the ninety-fifth percentile, and 17.70% at the ninety-ninth percentile, while flow success decreases by 3.63 percentage points. This non-uniform movement produces a deformed, heavy-tailed distribution rather than a uniform location shift.

The request population also develops a bimodal latency distribution with a secondary concentration near the run-specific 45-second step timeout. Successful Integrating the Healthcare Enterprise IT Infrastructure transaction 18 flows reach 37,869.5 ms at the ninety-fifth percentile and 43,972.1 ms at the ninety-ninth percentile. Their upper tail therefore lies in the same tens-of-seconds regime, although the flow and request populations remain distinct.

Akram et al. define the Load-Sharing Window (LSW) as the stochastic interval within which state remains useful for distributed load transfer. As traffic intensity and communication delay rise, this window narrows until a transfer arrives too late to produce useful work, a condition termed Transfer in Vain (TIV)[2].

The experiment does not observe a regional load-transfer protocol and does not claim literal identity with that topology. It observes the homologous application-level signature: admission stays saturated, delivery drift expands, the latency tail grows, and the proportion of useful terminal completions falls. These converging signals are consistent with a closing load-sharing window at the client boundary.

The timeout-bound mode marks the practical transfer-in-vain risk boundary for this workload. An occurrence that retains a local task, connection, buffer, or semaphore position while its transaction approaches the configured 45-second

timeout consumes transport and execution resources without guaranteeing useful completion before the configured limit.

Transaction 18 tail values close to this limit show how client resources remain committed to work that is at increasing risk of termination. The evidence supports this risk interpretation, rather than a claim that every timed-out request is provably futile at dispatch time. Taken together, semaphore occupancy, delivery drift, outcome-conditioned latency, and useful completion make the closing window empirically testable at the client boundary.

4.2 Limitations of the Empirical Evidence

4.2.1 Overview

The conclusions remain conditional on the observation perimeter, workload construction, and execution platform of the campaign. These boundaries do not invalidate the measurements; they define the population and causal claims to which those measurements apply. Four limitations are particularly important for interpreting the path-level overhead, the saturation signature, and the external validity of the method.

Black-Box Observability Boundary

The regional integration path remains a black box to the experiment. Internal logs from the Application Profile Management System (APMS), gateway components, and downstream services are inaccessible within the authorised security and policy perimeter. The Request Dataset Generator (RDG) therefore measures the interval between client dispatch and client receipt and reconstructs state from client-visible outcomes.

The method measures a declared path differential, but it cannot partition that differential among network transport, connection establishment, security processing, queue residence, transformation, routing, gateway computation, and downstream execution. Consequently, $\Delta L(P_{50}) = 184.7$ *ms remains an aggregate surrogate displacement*, not a direct measurement of gateway service

time.

A causal decomposition requires aligned ingress, component-level, downstream, and egress timestamps under a common correlation identifier and a quantified clock-error bound. Without that instrumentation, any allocation of the displacement to one internal phase remains unsupported.

Workload and Semantic Scope

The workload reflects the Veneto Sistema Informativo Territoriale (SI.Ter) integration perimeter represented in the available specifications and current implementation. It exercises Anagrafe Zero patient interactions, Simple Object Access Protocol (SOAP) document transactions including ITI-18 and ITI-43, and Scryba Sign operations governed by the available developer guide[3].

These profiles make the experiment domain-specific at the level of the represented semantics: security prerequisites, patient identities, document references, and signature outputs remain ordered rather than collapsing into independent endpoint calls. They do not establish operational representativeness or define a universal healthcare workload.

The measured stochastic distributions cannot be transferred without validation to purely Representational State Transfer (REST) systems, GraphQL interfaces, message-broker topologies, or fully decoupled event-driven microservices. Those architectures change connection reuse, synchronisation, backpressure, state ownership, and completion semantics.

Local Client Ingestion Bottlenecks

The extreme campaign also profiles the limits of a single load-generator node. Once active flows occupy the semaphore, queued occurrences accumulate delivery drift and the realised input to the remote system diverges from the con-

[3]Vio et al., *Specifiche tecniche – Anagrafe Zero*; CNR (ICAR) e Dipartimento per la Trasformazione Digitale, *Specifiche tecniche per l'interoperabilità tra i sistemi regionali di FSE: Affinity Domain Italia*; Medas S.r.l., *Scryba Sign 3.x Developer's Guide*.

figured arrival process. The observed plateau and drift therefore demonstrate *local admission saturation*, but they do not isolate one machine-level cause. Event-loop scheduling, operating-system context switching, socket availability, memory pressure, logging, and flow-level semaphore waiting can all contribute. This confounding effect limits the offered rate λ that the experiment can attribute exclusively to the system under test.

Above the local knee, an increase in scheduled traffic can increase client-side queueing without increasing remotely delivered traffic by the same amount. The extreme run consequently characterises the coupled generator–path system after local saturation rather than an isolated provider capacity.

Replication and Population Balance

A cross-cutting sample limitation reinforces these three boundaries. The final evidence set contains 74 complete live runs from a protocol that specifies 300, and the number of complete repetitions differs across scenarios. The large number of request and flow occurrences improves descriptive precision for the observed populations, but it does not replace balanced independent replication. The campaign therefore supports empirical characterisation and model validation within its realised sample. It does not establish a universal capacity threshold, a service-level guarantee, or production-wide traffic representativeness.

4.3 Future Directions

4.3.1 Engineering Scalability and Platform Evolutions

The engineering roadmap separates generator capacity from gateway capacity by scaling, controlling, and grounding the experimental instrument. The three evolutions below address, respectively, distributed workload generation, closed-loop load control, and trace-based workload construction. Each retains the thesis's evidence discipline by making calibration conditions, observation boundaries, and failure states explicit.

Distributed Workload Generation

The first evolution replaces the single-node Request Dataset Generator (RDG) with a Kubernetes-orchestrated cluster whose workers execute disjoint partitions of one global occurrence space. A control plane assigns deterministic seed intervals, immutable configuration digests, globally unique occurrence identifiers, and a common experiment epoch. Each worker maintains a local asynchronous pool and publishes occurrence-scoped events to a partitioned collector. If worker k delivers intensity $\lambda_k(t)$, the intended offered load is

$$\lambda(t) = \sum_{k=1}^{n} \lambda_k(t).$$

This identity is accepted empirically only when per-worker scheduling drift remains below a calibration bound and remotely observed arrivals grow proportionally with n.

A two-phase start barrier separates image and connection warm-up from timed execution. Monotonic clocks measure local durations, while a clock-synchronisation service supplies an uncertainty envelope for cross-node ordering. The manifest records worker placement, clock error, image version, seed range, and partial-failure state in addition to the present run metadata. The design removes the single event loop as the immediate offered-load bottleneck, supports geographically distinct entry points, and preserves the runtime-level distinction between concurrency, distribution, and cloud scalability[4].

[4]Università degli Studi di Padova. *Runtimes for Concurrency and Distribution.* Course syllabus SCQ0093640. Academic year 2026/2027; responsible instructor: Tullio Vardanega. Università degli Studi di Padova, Aug. 24, 2026; Apache Software Foundation, *Apache JMeter User's Manual: Remote (Distributed) Testing*.

Adaptive Closed-Loop Workload Control

The second evolution turns the open-loop scheduler into a constrained feedback experiment. At control instant k, the controller observes

$$y_k = \left(P_{95}^{\text{drift}}, P_{95}^{\text{latency}}, e_k, X_k, C_k\right),$$

where e_k is the error rate, X_k is useful-flow throughput, and C_k is admitted concurrency. It manipulates either the next offered rate or the admission bound. A Proportional–Integral–Derivative (PID) law can compute

$$u_k = K_p \varepsilon_k + K_i \sum_{h=0}^{k} \varepsilon_h \Delta t + K_d \frac{\varepsilon_k - \varepsilon_{k-1}}{\Delta t}, \tag{4.5}$$

with saturation, anti-windup, cool-down intervals, and explicit request and error budgets.

Change-point logic marks a candidate stochastic knee when $\partial X/\partial \lambda$ approaches zero while the derivatives of tail latency, delivery drift, or failures remain positive. Repeated approaches from below, under independent seeds and a fixed semantic mix, produce a confidence region for that knee rather than an unsupported exact gateway threshold. A Reinforcement Learning agent can replace the fixed controller when the response surface is history-dependent, but it trains on mock or isolated traces and operates live only behind a safety shield that constrains its action space. The controller also checks per-worker headroom, because a feedback law that observes only end-to-end latency can misclassify generator saturation as remote saturation.

Anonymised Real-World Trace Replay

The third evolution constructs an authorised trace-ingestion and replay pipeline for regional clinical traffic. Parsers map heterogeneous operational records to a canonical schema; privacy transformations remove unnecessary payloads, tokenise identifiers consistently within one authorised trace, generalise rare at-

tributes, and shift absolute timestamps while preserving inter-arrival intervals. A dependency-reconstruction stage then recovers causal order, retries, flow mix, payload-size classes, and prerequisite relationships before compiling the trace into RDG occurrences and a non-stationary arrival timeline $\lambda(t)$.

Replay fidelity is assessed through approved aggregate properties— including autocorrelation, burst duration, concurrency, and flow proportions— and not through access to identifying clinical content. A segment whose initial state or dependencies remain incomplete is rejected rather than converted into an independent request stream, because accurate scaling preserves both temporal and causal structure[5]. The pipeline keeps the re-identification secret outside the experimental environment, records an auditable transformation manifest, and applies data minimisation and disclosure-risk review before any replay artefact enters the cluster[6].

4.3.2 Theoretical Computer Science Perspectives

The theoretical roadmap converts observed regularities into proposed proof obligations. These directions extend the empirical contribution; they do not report proofs completed by this thesis. Each obligation therefore remains conditional on an explicit abstraction, an identified population, and a declared observation boundary.

Predictable Dispatch and Release Jitter

The first proposed extension treats the divergence represented in Figure 4.2 as a local schedulability problem. The non-stationary Poisson generator admits arbitrarily short inter-arrival distances, so a regulator first maps each flow class to a sporadic task $\tau_i = (C_i, T_i, D_i, J_i)$ with worst-case local demand C_i, minimum

[5]Zhu, Chen, and Chiueh, "TBBT: Scalable and Accurate Trace Replay for File Server Evaluation".

[6]European Parliament and Council of the European Union, *Regulation (EU) 2016/679 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data*.

inter-arrival time T_i, deadline D_i, and release jitter J_i. Under the classical independent, implicit-deadline assumptions, the sufficient utilisation bounds for Rate Monotonic and Earliest Deadline First scheduling on one processor are, respectively,

$$\sum_i \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right), \qquad \sum_i \frac{C_i}{T_i} \leq 1.$$

Shared resources require the response-time recurrence

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i + J_j}{T_j} \right\rceil C_j, \qquad R_i + J_i \leq D_i, \tag{4.6}$$

where B_i bounds local blocking and $hp(i)$ contains the higher-priority tasks. The analytical target for the lower chronogram is $J_i^{dispatch} = \max(0, s_i - r_i) \leq \epsilon_i$, with release time r_i, actual start s_i, and a declared engineering bound ϵ_i; it is not a measured microsecond guarantee. A qualified hard real-time platform can host the timing-critical dispatcher, while the `PREEMPT_RT` real-time-preemption configuration provides an intermediate Linux option. The Priority Ceiling Protocol can bound inversion in local credential stores, connection pools, and log buffers used by mutual Transport Layer Security and Security Assertion Markup Language processing. These mechanisms bound local scheduling effects, not the remote network itself[7].

Static Verification of Concurrency Bounds

The second proposed extension applies abstract interpretation to the asynchronous executor's lowered control-flow graph. Let Σ be the concrete state space and let $F(X) = \Sigma_0 \cup \mathrm{post}(X)$ be the monotone collecting transformer. Under the required ω-continuity assumption, its concrete collecting semantics

[7]Università degli Studi di Padova. *Real-Time Kernels and Systems.* Course syllabus SCQ0093641. Academic year 2024/2025; instructor: Tullio Vardanega. Università degli Studi di Padova, Aug. 24, 2026; Jane W. S. Liu. *Real-Time Systems.* Prentice Hall, 2000. ISBN: 9780130996510; Alan Burns and Andy Wellings. *Analysable Real-Time Systems: Programmed in Ada.* CreateSpace Independent Publishing Platform, 2016. ISBN: 9781530265503.

is

$$\mathcal{S}_{conc} = \mathrm{lfp}(F) = \bigcup_{n \in \mathbb{N}} F^n(\varnothing). \tag{4.7}$$

A Galois connection

$$\langle \mathcal{P}(\Sigma), \subseteq \rangle \underset{\gamma}{\overset{\alpha}{\rightleftarrows}} \left\langle D^\sharp, \sqsubseteq \right\rangle \tag{4.8}$$

maps concrete states to an interval domain $D^\sharp_{Int}$ and a relational octagon domain $D^\sharp_{Oct}$.

A sound transformer $F^\sharp$ computes a post-fixpoint through widening,

$$X^\sharp_{k+1} = X^\sharp_k \nabla F^\sharp(X^\sharp_k), \tag{4.9}$$

and aims to infer, for active flows a, held permits h, and available permits p, the inductive invariant

$$0 \le a \le h \le 30, \qquad 0 \le p \le 30, \qquad h + p = 30. \tag{4.10}$$

If the analyser establishes this invariant for normal completion, exceptions, and cancellation, soundness yields $a \le 30$ for every arrival trace.

This proof remains future work. It also does not bound waiting tasks: the present scheduler can create an unbounded waiting population, so proving $0 \le q \le Q$ additionally requires a bounded producer queue and an explicit block, shed, or defer policy[8].

[8] Università degli Studi di Padova. *Software Verification*. Course syllabus SCQ0089515. Academic year 2026/2027; responsible instructor: Francesco Ranzato. Università degli Studi di Padova, Aug. 24, 2026; Antoine Miné. "Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation". In: *Foundations and Trends in Programming Languages* 4.3–4 (2017), pp. 120–372. DOI: 10.1561/2500000034.

Bibliography

Akram, Saira, Muhammad Asim Akram, and Fattah UR Rehman. “Numerical Evaluation of Network Latency and Throughput in Distributed Systems”. In: *The Science Post* 1.4 (Dec. 2025). DOI: 10.5281/zenodo.18012513 (cit. on pp. 27, 29, 80, 88).

Azienda ULSS 3 Serenissima. *Specifiche tecniche di integrazione tra sistemi Document Source e Repository XValue*. Technical specification. Version 2.9. Azienda ULSS 3 Serenissima, Aug. 5, 2024.

Azienda Zero. *Data Management Reference Architecture: Documento descrittivo della reference architecture per il data management*. Reference architecture. Version 1.1. Azienda Zero, Sept. 19, 2025.

— *Infrastrutture – Reference Architecture: Architettura di riferimento dell’infrastruttura di Azienda Zero*. Reference architecture. Version 1.1.0. Azienda Zero, Oct. 9, 2025.

— *Realizzazione di un Sistema Informativo Territoriale per gli Enti Sanitari della Regione Veneto*. Technical specifications. Specific procurement, Digital Healthcare – Clinical-Care Information Systems, ID 2365, Lot 3; date derived from file metadata. Azienda Zero, July 9, 2025 (cit. on pp. 3, 33, 46).

Bergamasco, Stefano. “Architettura di un Sistema Informativo Sanitario (SIS), con focus particolare su sistemi di Datawarehouse e flussi informativi”. Materiale didattico per il corso Funzioni direttive dei servizi sanitari e formazione manageriale per le direzioni generali. Bologna, May 21, 2025.

Bergamasco, Stefano and Nicola Rodeghiero. *Documento di dimensionamento computazionale: Realizzazione di un Sistema Informativo Territoriale per gli Enti Sanitari della Regione Veneto*. Sizing report. Version 1.4. Raggruppamento Temporaneo di Imprese SI.Ter, June 26, 2026.

Bianchi, Marzia. *Firme e sigilli elettronici: Guida sintetica*. Technical presentation. Date and author derived from the metadata of the consulted file. Mar. 23, 2026.

Burns, Alan and Andy Wellings. *Analysable Real-Time Systems: Programmed in Ada*. CreateSpace Independent Publishing Platform, 2016. ISBN: 9781530265503 (cit. on p. 97).

Canal, Gregorio and Matteo Cognolato. *Specifiche tecniche – Gestione sottoscrizione/notifica*. Technical specification. Version 2.2. Consorzio Arsenàl.IT, Jan. 22, 2021.

Canal, Gregorio and Gianluca Pavan. *Specifiche tecniche chiamata di contesto RVE*. Technical specification. Version 1.3. Consorzio Arsenàl.IT, July 18, 2025 (cit. on pp. 11, 34).

CNR (ICAR) e Dipartimento per la Trasformazione Digitale. *Specifiche tecniche per l'interoperabilità tra i sistemi regionali di FSE: Affinity Domain Italia*. Technical specification. Version 2.6.3. CNR (ICAR) e Dipartimento per la Trasformazione Digitale, Oct. 31, 2025 (cit. on pp. 5, 35, 92).

Cousot, Patrick and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints". In: *Proceedings of the Fourth Annual Symposium on Principles of Programming Languages*. Association for Computing Machinery, 1977, pp. 238–252. DOI: 10.1145/512950.512973.

Dimitrov, Rossen and Anthony Skjellum. *Impact of Latency on Applications' Performance*. Technical Report. Systems Research Group, Northwestern University, Apr. 2001. URL: https://users.cs.northwestern.edu/~srg/Papers/01-10-02/srg_2.pdf (visited on 07/31/2026) (cit. on pp. 29, 30).

DXC Technology. *Specifiche di integrazione autenticazione APMS*. Technical specification. Version 1.4.1. Azienda Zero, Nov. 22, 2024 (cit. on pp. 4, 11, 23).

Esposito, Andrea. *Realizzazione integrazioni per progetto SI.Ter: Specifiche di integrazione degli scenari di Anagrafica*. Technical specification. Version 1.0. Raggruppamento Temporaneo di Imprese SI.Ter, May 27, 2026.

European Parliament and Council of the European Union. *Regulation (EU) 2016/679 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data*. Regulation (EU) 2016/679. Official Journal of the European Union, L 119. European Union, Apr. 27, 2016, pp. 1–88. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj (visited on 07/29/2026) (cit. on pp. 2, 96).

Integrazione tra Advenias e Paymed – Rendicontazione Accessi. Project technical note. Date derived from the metadata of the consulted file; author not identified. June 11, 2025.

Iorio, Marco, Fulvio Risso, and Claudio Casetti. "When Latency Matters: Measurements and Lessons Learned". In: *ACM SIGCOMM Computer Communication Review* 51.4 (Oct. 2021), pp. 2–13. DOI: 10.1145/3503954.3503956 (cit. on pp. 25, 29, 80, 85).

Lamport, Leslie. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ISBN: 9780321143068.

Liu, Jane W. S. *Real-Time Systems*. Prentice Hall, 2000. ISBN: 9780130996510 (cit. on p. 97).

MacKenzie, C. Matthew et al. *Reference Model for Service Oriented Architecture 1.0*. Committee Specification. OASIS, Oct. 12, 2006. URL: https://docs.oasis-open.org/soa-rm/v1.0/soa-rm.html (visited on 07/22/2026).

Medas S.r.l. *Scryba Sign 3.x Developer's Guide*. Developer's guide. Version 16. Restricted internal document. Medas S.r.l., Apr. 10, 2024 (cit. on pp. 4, 38, 39, 58, 92).

Miné, Antoine. "Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation". In: *Foundations and Trends in Programming Languages* 4.3–4 (2017), pp. 120–372. DOI: 10.1561/2500000034 (cit. on p. 98).

Ministero della Salute. *Fascicolo sanitario elettronico 2.0*. Ministerial decree Decreto 7 settembre 2023. Italian Official Gazette, General Series no. 249, 24 October 2023, as subsequently amended. Ministero della Salute, Sept. 7, 2023. URL: https://www.gazzettaufficiale.it/eli/id/2023/10/24/23A05829/sg (visited on 07/29/2026) (cit. on p. 3).

— *Regolamento recante la definizione di modelli e standard per lo sviluppo dell'assistenza territoriale nel Servizio sanitario nazionale*. Ministerial decree Decreto 23 maggio 2022, n. 77. Italian Official Gazette, General Series no. 144, 22 June 2022. Ministero della Salute, May 23, 2022 (cit. on pp. 1, 2).

Pastore, Claudio and Federica Martello. *Percorsi, Sistemi e Flussi Sanitari dei Servizi Territoriali: Dalla Governance ai Servizi: logiche, strumenti e interoperabilità*. Technical report. Version 3.0. Internal technical document. AlmavivA, Mar. 17, 2026.

Progetto SI.Ter: Presentazione alla Cabina di Regia. Project presentation. The consulted document does not identify an author. Mar. 25, 2026.

Raggruppamento Temporaneo di Imprese guidato da AlmavivA. *Sistema Informativo Territoriale per gli Enti Sanitari della Regione del Veneto – Lotto 3*. Technical report. Date derived from file metadata. Raggruppamento Temporaneo di Imprese guidato da AlmavivA, Sept. 19, 2025 (cit. on pp. 3, 4, 6, 9, 33).

Regione del Veneto. *Progetto sistema di risposta sanitaria 116117: Istituzione delle relative centrali operative*. Project document. Revision date derived from the filename of the consulted document. Regione del Veneto, June 18, 2024.

Regione del Veneto, Area Sanità e Sociale. *Modello di Governo per l'implementazione del Sistema Informativo Territoriale (SI.Ter) nelle Aziende ed Enti del Servizio Sanitario Regionale della Regione del Veneto*. Regional decree Decreto n. 11922. Regione del Veneto, Mar. 16, 2026 (cit. on p. 3).

Regione del Veneto, Direzione Programmazione Sanitaria. *PNRR Missione 6, Componente 2, Investimento 1.3.2: Approvazione del tracciato definitivo del flusso informativo sanitario SIOC*. Regional decree Decreto n. 13745. Regione del Veneto, Dec. 17, 2025.

Regione del Veneto, Gruppo tecnico di lavoro 116117. *116117: Manuale – Protocollo 2*. Operating manual. The consulted document does not indicate a publication date.

Università degli Studi di Padova. *Real-Time Kernels and Systems*. Course syllabus SCQ0093641. Academic year 2024/2025; instructor: Tullio Vardanega. Università degli Studi di Padova, Aug. 24, 2026 (cit. on p. 97).

— *Runtimes for Concurrency and Distribution*. Course syllabus SCQ0093640. Academic year 2026/2027; responsible instructor: Tullio Vardanega. Università degli Studi di Padova, Aug. 24, 2026 (cit. on p. 94).

— *Software Verification*. Course syllabus SCQ0089515. Academic year 2026/2027; responsible instructor: Francesco Ranzato. Università degli Studi di Padova, Aug. 24, 2026 (cit. on p. 98).

Vio, Elena et al. *Specifiche tecniche – Anagrafe Zero*. Technical specification. Version 2.6. Consorzio Arsenàl.IT, Jan. 9, 2024 (cit. on pp. 5, 9, 33, 34, 92).

Zanardini, Mauro et al. *Infrastruttura di sicurezza GDL-O Sicurezza*. Technical specification. Version 2.14. Consorzio Arsenàl.IT, June 28, 2024 (cit. on pp. 9, 11, 23, 34).

Zhu, Ningning, Jiawu Chen, and Tzi-cker Chiueh. "TBBT: Scalable and Accurate Trace Replay for File Server Evaluation". In: *4th USENIX Conference on File and Storage Technologies (FAST 05)*. San Francisco, CA: USENIX Association, Dec. 2005. URL: https://www.usenix.org/conference/fast-05/tbbt-scalable-and-accurate-trace-replay-file-server-evaluation (visited on 07/29/2026) (cit. on pp. 19, 96).

Online Sources

Apache Software Foundation. *Apache JMeter User's Manual: Component Reference*. URL: https://jmeter.apache.org/usermanual/component_reference.html (visited on 08/04/2026) (cit. on p. 47).

— *Apache JMeter User's Manual: Elements of a Test Plan*. URL: https://jmeter.apache.org/usermanual/test_plan.html (visited on 08/04/2026) (cit. on p. 47).

— *Apache JMeter User's Manual: Functions and Variables*. URL: https://jmeter.apache.org/usermanual/functions.html (visited on 08/04/2026) (cit. on p. 47).

— *Apache JMeter User's Manual: Remote (Distributed) Testing*. URL: https://jmeter.apache.org/usermanual/remote-test.html (visited on 08/04/2026) (cit. on pp. 47, 94).

European Parliament and Council of the European Union. *Regulation (EU) No 910/2014 on Electronic Identification and Trust Services for Electronic Transactions in the Internal Market*. Consolidated text of 18 October 2024. July 23, 2014. URL: https://eur-lex.europa.eu/eli/reg/2014/910/2024-10-18/eng (visited on 08/05/2026) (cit. on p. 4).

HL7 International. *FHIR Release 4 (Version 4.0.1): Overview*. Nov. 1, 2019. URL: https://hl7.org/fhir/R4/overview.html (visited on 07/22/2026).

— *FHIR Release 4 (Version 4.0.1): Resource Bundle*. Nov. 1, 2019. URL: https://hl7.org/fhir/R4/bundle.html (visited on 07/22/2026).

IHE International. *IHE IT Infrastructure Technical Framework, Volume 1: Cross-Enterprise Document Sharing (XDS.b), Revision 20.2*. Nov. 11, 2025. URL: https://profiles.ihe.net/ITI/TF/Volume1/ch-10.html (visited on 07/22/2026) (cit. on pp. 5, 35).

IHE IT Infrastructure Technical Committee. *Internet User Authorization (IUA), Revision 2.5*. June 18, 2026. URL: https://profiles.ihe.net/ITI/IUA/ (visited on 07/22/2026).

Santoni, Adriano. *RFC 5544: Syntax for Binding Documents with Time-Stamps*. RFC Editor. Feb. 2010. DOI: 10.17487/RFC5544. URL: https://www.rfc-editor.org/info/rfc5544/ (visited on 08/05/2026) (cit. on p. 4).